Supplementary Information

CUQDyn1_Plus: Hybrid Conformal–Gaussian Uncertainty Quantification for Partially Observed Dynamical Systems

A. Portela J. R. Banga

Generated 2026-07-20

Abstract

This Supplementary Information consolidates the full computational evaluation of the CUQDyn1_Plus toolbox into a single document. It integrates three automatically generated reports—the combined problem-definition and three-method UQ tutorial, the empirical results across the maintained benchmark models, and the CUQDyn1_Plus-versus-PyMC ABC-SMC comparison—together with additional appendices that document per-run settings, UQ identifiability diagnostics, Bayesian convergence diagnostics, HybridCov covariance diagnostics, an analytic LinearCascade3 non-identifiability case study, per-model method notes, and scope limitations. All figures and tables use supplementary (S-prefixed) numbering. Every artifact was produced by a single end-to-end run (`scripts/run_full_evaluation.m`); Part IV collects the additional diagnostics and the exact per-run reproduction parameters.

Contents

I Problem definition and three-method UQ tutorial **4**

S1 Problem Definition Workflow **4**
S1.1 CUQDyn1_Plus Problem Definition Tutorial 4

S2 Prediction-UQ Workflow **10**
S2.1 CUQDyn1_Plus LV Tutorial: Three Prediction-UQ Workflows 10

S3 Representative Figures From Example Results **15**

II Empirical results across benchmark models **19**

S4 Scope **19**

S5 Trajectory Fit and Prediction-UQ Summaries **21**

S6 LV Three-Method Tutorial **24**

S7 Linear Cascade Known-Truth Checks **24**

S8 Simulation-Based Calibration **24**

S9 Example Figures **28**

S10 Notes and Caveats **28**

III CUQDyn1_Plus versus PyMC ABC-SMC **37**

S11 Scope and Provenance **37**

S12 Example Settings **38**

S13 Method Settings **38**
S13.1 CUQDyn1_Plus Settings 38
S13.2 PyMC ABC-SMC Settings 39

S14 High-Level Parameter Comparison **41**

S15 High-Level Trajectory-Band Comparison **42**

S16 Side-by-Side Predictive Figures **43**

S17 Full Parameter Table **48**
S17.1 LV 48
S17.2 SIR 48
S17.3 AP 48
S17.4 NFKB 49

S18 Full Trajectory-Band Table **51**
S18.1 LV 51
S18.2 SIR 51
S18.3 AP 52
S18.4 NFKB 52

S19 Detailed Discussion of the Updated Comparison **54**

IV Additional diagnostics and reproducibility **56**

S20 UQ identifiability diagnostics **56**

S21 HybridCov covariance diagnostics **56**
S21.1 LV 57
S21.2 SIR 59

S22 A non-identifiability failure mode revealed by SBC (LinearCascade3) **63**

S23 Bayesian ABC-SMC diagnostics **63**

S24 Optimizer, ODE, cost and FIM settings per run **65**

S25 Per-model method notes **70**

S25.1 Lotka-Volterra (LV) 70
S25.2 SIR 70
S25.3 Alpha-pinene (AP) 70
S25.4 NF-κB 71
S25.5 LinearCascade 71

S26 Coverage and scope limitations **72**

Part I. Problem definition and three-method UQ tutorial

S1 Problem Definition Workflow

S1.1 CUQDyn1_Plus Problem Definition Tutorial

This tutorial shows how to set up a new ODE problem from a high-level MATLAB definition. The goal is to let new users define a model once, then generate the legacy problem files and optional synthetic-data script used by CUQDyn1_Plus.

For the separate tutorial on FIM, HybridCov, and bootstrap prediction uncertainty workflows, see `tutorialUQ.md`.

S1.1.1 Overview

A CUQDyn problem definition can generate:

1. `prob_mod_dynamics_<Name>.m`
2. `prob_mod_cost_<Name>.m`
3. `generateSyntheticData_<Name>.m`

The generated dynamics/cost files keep the existing CUQDyn interface unchanged. The generated data script writes CSV files in the same convention used by `loadStateData`:

- column 1 is time;
- columns 2:end are states;
- all states are finite at `t=0`;
- observed states are finite after `t=0`;
- hidden states are `NaN` after `t=0`.

S1.1.2 Start From The Template

Copy the cheatsheet:

```
copyfile('EXAMPLES/problem_definition_template.m', ...
    'EXAMPLES/MyModel/define_problem_MyModel.m')
```

Then edit the copy. The required fields are:

```
problem.name = "MyModel";
problem.states = ["x1"; "x2"];
problem.parameters = ["a"; "b"; "c"];
problem.odes = [
    "a*x1 - b*x1*x2"
    "b*x1*x2 - c*x2"
];
```

State and parameter names must be valid MATLAB identifiers because the generator uses them directly in the generated dynamics function.

S1.1.3 Add Fitting Metadata

Declare guesses and bounds:

```
problem.initial_guess = [0.5; 0.02; 0.4];
problem.parameter_bounds.lower = [0.01; 0.001; 0.01];
problem.parameter_bounds.upper = [5; 1; 5];
problem.observed_states = "x2";
problem.observed_state_indices = 2;
```

The observed-state names are useful for readability. The indices are useful for data generation and checking.

S1.1.4 Declare Residual Scaling

If observed states have different units or magnitudes, declare how residuals should be normalized. For known measurement standard deviations:

```
problem.cost.residual_model = "known_sigma";
problem.cost.sigma_mode = "explicit";
problem.cost.sigma = 0.1;          % one value per observed state
problem.cost.sigma_is_known = true;
```

For synthetic data generated with a noise percentage:

```
problem.cost.residual_model = "known_sigma";
problem.cost.sigma_mode = "from_reference_trajectory_mean";
problem.cost.noise_percent = 10;
problem.cost.sigma_is_known = true;
```

For raw unweighted residuals:

```
problem.cost.residual_model = "none";
```

For manual deterministic scaling:

```
problem.cost.residual_model = "state_weights";
problem.cost.observed_state_weights = [1/0.1, 1/25.0];
```

At run time, convert this declaration to CUQDyn cost options:

```
cost_opts = cuqdyn_cost_options_from_problem(problem);
```

If `sigma_mode` is `"from_reference_trajectory_mean"`, provide the reference trajectory and observed-state indices:

```
cost_opts = cuqdyn_cost_options_from_problem(problem, Y_true, observed_idx);
```

S1.1.5 Declare Synthetic Data Settings

If you want generated benchmark data, add `problem.synthetic_data`:

```
problem.synthetic_data.times = linspace(0, 20, 41)';
problem.synthetic_data.initial_values = [10; 5];
problem.synthetic_data.true_parameters = [0.5; 0.02; 0.4];
problem.synthetic_data.observed_state_indices = 2;
problem.synthetic_data.noise_model = "additive_gaussian_mean_percent";
```

```
problem.synthetic_data.noise_percent = 10;
problem.synthetic_data.rng_seed = 101;
problem.synthetic_data.min_observed_value = 0;
problem.synthetic_data.output_folder = "data";
problem.synthetic_data.output_file = "my_model_data.csv";
problem.synthetic_data.make_plots = true;
```

`min_observed_value = 0` prevents negative noisy finite observations while preserving exact zero initial conditions. Use a small positive value only if your measurement model requires strictly positive observations after `t=0`.

S1.1.6 Using Real Data

If you already have experimental or observational data, provide it as a CSV file in the format expected by `loadStateData`.

The CSV must contain:

- column 1: time;
- columns 2:end: model states, in the same order as `problem.states`;
- row 1: initial values for **all** states;
- after row 1: finite values for observed states;
- after row 1: `NaN` for hidden or unobserved states.

For a three-state model where only state `x2` is observed after the initial condition:

```
time,x1,x2,x3
0,10,0,0
1,NaN,1.23,NaN
2,NaN,2.05,NaN
3,NaN,2.61,NaN
```

Then declare the data in your problem file:

```
problem.data.folder = "data";
problem.data.file = "my_real_data.csv";
problem.observed_states = "x2";
problem.observed_state_indices = 2;
```

and load it in the run script:

```
[times, all_state_data, ic, observed_data, observed_idx] = ...
    loadStateData(fullfile(exampleDir, problem.data.folder), ...
    problem.data.file, numel(problem.states));
```

Real-Data Error Models The residual/error model matters because it controls parameter fitting and FIM covariance scaling. If observed states have different magnitudes, raw residuals can make the largest-magnitude state dominate the fit.

If you know the measurement standard deviation for each observed state, use `known_sigma`:

```
problem.cost.residual_model = "known_sigma";
problem.cost.sigma_mode = "explicit";
problem.cost.sigma = [0.1, 25.0];   % one sigma per observed state
problem.cost.sigma_is_known = true;
```

This is usually the best option when the instrument, assay, or preprocessing pipeline provides uncertainty estimates.

If you do not know the measurement error model, there is no single automatic answer. Reasonable starting points are:

```
% Raw least squares: use only when observed states have comparable scales.
problem.cost.residual_model = "none";
```

or manual scale normalization:

```
% Example: divide residuals by representative state scales.
problem.cost.residual_model = "state_weights";
problem.cost.observed_state_weights = [1/0.1, 1/25.0];
```

Choose state weights from defensible scales such as typical measurement standard deviations, replicate variability, baseline-normalized units, or a representative range/mean for each observed state. Treat these as modeling assumptions and report them with the fit. For serious applications, compare sensitivity to plausible weighting choices and inspect residual diagnostics.

S1.1.7 Validate And Generate Files

From the repository root:

```
setPath_CUQDyn1_Plus
problem = define_problem_MyModel();
cuqdyn_validate_problem(problem);

exampleDir = fullfile(pwd, 'EXAMPLES', 'MyModel');
generatedDir = fullfile(exampleDir, 'generated_problem');

cuqdyn_generate_problem_files(problem, generatedDir, 'Overwrite', true);
cuqdyn_generate_data_script(problem, generatedDir, 'Overwrite', true);
```

Generated `generated_problem/` folders are disposable local outputs and are ignored by Git. Keep `define_problem_<Name>.m` as the source of truth; regenerate the legacy-compatible model files whenever you need to inspect or run them.

S1.1.8 Generate A CSV

Run the generated data script:

```
cd EXAMPLES/MyModel/generated_problem
generateSyntheticData_MyModel
```

Or generate the CSV directly:

```
addpath(generatedDir, '-begin');
out = cuqdyn_generate_synthetic_data(problem, ...
    'BaseDir', exampleDir, ...
    'DynamicsHandle', @prob_mod_dynamics_MyModel);
```

Then check the CSV with the standard loader:

```
[times, all_state_data, ic, observed_data, observed_idx] = ...
    loadStateData(fullfile(exampleDir, 'data'), ...
    problem.synthetic_data.output_file, numel(problem.states));
```

S1.1.9 Use The Generated Problem In A Run

```
addpath(generatedDir, '-begin');

dynamics_handle = @prob_mod_dynamics_MyModel;
cost_handle = @prob_mod_cost_MyModel;

nstates = numel(problem.states);
n_params = numel(problem.parameters);
guess_params = problem.initial_guess(:)';
lb_params = problem.parameter_bounds.lower(:)';
ub_params = problem.parameter_bounds.upper(:)';
alp = 0.025;

opts = cuqdyn_default_options(n_params);
opts.cost = cuqdyn_cost_options_from_problem(problem);
meigo_opts = opts.meigo;
meigo_opts.cost_opts = opts.cost;

resultDir = "Results_MyModel_" + string(datetime('now'), 'yyyy-MM-dd_HH-mm-ss');
mkdir(resultDir);

results = CUQDyn1_Plus(cost_handle, dynamics_handle, nstates, n_params, ...
    guess_params, lb_params, ub_params, alp, ...
    times, all_state_data, ic, observed_data, observed_idx, resultDir, ...
    meigo_opts);
```

For synthetic data with `sigma_mode = "from_reference_trajectory_mean"`, compute `Y_true` first and call:

```
opts.cost = cuqdyn_cost_options_from_problem(problem, Y_true, observed_idx);
```

S1.1.10 Check Built-In Examples

The maintained examples include high-level definitions. From the repository root, run:

```
reports = check_generated_problem_definitions();
```

This regenerates dynamics/cost files in temporary folders and verifies that generated outputs match the current legacy files.

S1.1.11 Regression Test The Generator

The toolbox also includes an automated regression test for the high-level problem-definition workflow:

```
results = runtests('unitTests/test_problem_definition_generator.m');
table(results)
```

This test is intended for developers and advanced users who modify the problem schema, generator, cost handling, or example definitions. It checks that:

- generated dynamics and cost modules for LV, SIR, AP, NF-kB, LinearCascade, and LinearCascade3 match the maintained legacy files;
- generated synthetic CSV files can be read by `loadStateData`;
- generated CSV files have the expected observed-state columns;
- finite generated data values are nonnegative.

Passing this test does not prove that a new scientific model is correct, but it does protect the generator interface from silent regressions.

S1.1.12 Practical Guidance

- Keep `define_problem_<Name>.m` as the single source of truth.
- Use `known_sigma` when observation standard deviations are known.
- Use `state_weights` only for deterministic normalization when no statistical sigma model is available.
- Treat `generated_problem/` as disposable local output. It is ignored by Git.
- Keep committed generated snapshots only when they are deliberately documented as references, such as `EXAMPLES/generated_reference/LV/`.

S2 Prediction-UQ Workflow

S2.1 CUQDyn1_Plus LV Tutorial: Three Prediction-UQ Workflows

This tutorial summarizes the Lotka-Volterra example in `EXAMPLES/LV/tutorial_LV_three_prediction_UQ_methods.m`. It is the cheapest example for comparing the three prediction uncertainty workflows currently available in CUQDyn1_Plus:

1. FIM delta-method trajectory bands with `CUQDyn1_Plus`
2. HybridCov delta-method trajectory bands with `CUQDyn1_Plus_HybridCov`
3. Parametric bootstrap latent trajectory bands with `bootstrap_trajectory_uq`

The LV tutorial uses a partially observed predator-prey dataset where prey is observed and predator is unobserved after the initial condition.

The hand-written LV dynamics and cost files remain in place, but the same problem is also declared in `EXAMPLES/LV/define_problem_LV.m`. That high-level definition can regenerate legacy-compatible `prob_mod_dynamics_LV.m` and `prob_mod_cost_LV.m` files automatically. It also declares the synthetic-data settings needed to generate a CUQDyn-formatted CSV.

S2.1.1 Setup

From the repository root, start MATLAB and run:

```
setPath_CUQDyn1_Plus
cd EXAMPLES/LV
tutorial_LV_three_prediction_UQ_methods
```

MEIGO must be available either through the `MEIGO64_PATH` environment variable or by placing `MEIGO64-master/` in the repository root. The local `MEIGO64-master/` folder is intentionally ignored by Git.

To inspect or regenerate the LV problem modules without changing the tutorial workflow, run:

```
problem = define_problem_LV();
cuqdyn_validate_problem(problem);
cuqdyn_generate_problem_files(problem, fullfile(pwd, 'generated_problem'), ...
    'Overwrite', true);
check_generated_LV_problem
```

The check writes generated files to a temporary folder and compares their dynamics/-cost outputs with the legacy files. Per-example `generated_problem/` folders are disposable local outputs and are ignored by Git. A small committed LV snapshot is available under `EXAMPLES/generated_reference/LV/` for inspection. For a schema overview and residual-normalization options, see `EXAMPLES/problem_definition_template.m`.

To generate a readable LV data-generation script from the same definition:

```
problem = define_problem_LV();
cuqdyn_generate_data_script(problem, fullfile(pwd, 'generated_problem'), ...
    'Overwrite', true);
```

The generated script calls `cuqdyn_generate_synthetic_data`, writes the CSV using the same `loadStateData` convention as the bundled examples, and validates that finite generated data are nonnegative.

S2.1.2 Tutorial Settings

The tutorial script exposes two main knobs:

```
opts.meigo.maxeval = 1500;
boot_opts.n_boot = 50;
```

For a quick smoke test, these can be reduced, for example:

```
opts.meigo.maxeval = 500;
boot_opts.n_boot = 10;
```

Low bootstrap counts are useful only to verify that the workflow executes. Coverage and interval width should not be interpreted strongly from very small bootstrap runs.

Because the LV dataset is synthetic, the tutorial uses the same additive Gaussian observation-error model used by the data generator:

```
opts.cost.residual_model = 'known_sigma';
opts.cost.sigma = cuqdyn_synthetic_sigma_from_trajectory(...);
```

This means optimisation and FIM covariance are based on standardized residuals, while the metric tables still report raw RMSE/MAE/NRMSE in the original state units.

The same residual-normalization declaration is present in `define_problem_LV.m` as:

```
problem.cost.residual_model = "known_sigma";
problem.cost.sigma_mode = "from_reference_trajectory_mean";
problem.cost.noise_percent = 10;
```

S2.1.3 Method 1: FIM Delta-Method UQ

The FIM workflow is run with:

```
fim_results = CUQDyn1_Plus(cost_handle, dynamics_handle, nstates, n_params, ...
    guess_params, lb_params, ub_params, alp, ...
    times, all_state_data, ic, observed_data, observed_idx, fimDir, meigo_opts);
```

For observed states, CUQDyn1_Plus uses leave-one-out conformal prediction bands. For unobserved states, it uses local Gaussian delta-method bands based on the Fisher information matrix covariance estimate.

The tutorial then calls:

```
state_names = {'Prey', 'Predator'};
save_trajectory_nrmse_tables(fim_results, fimDir, state_names);
diagnose_uq_quality(fim_results, fimDir, param_names, state_names, lb_params, ub_params);
```

These functions produce compact Excel tables for trajectory fit quality, coverage, interval widths, residuals, covariance conditioning, parameter standard deviations, correlations, and near-bound diagnostics.

S2.1.4 Method 2: HybridCov Delta-Method UQ

The HybridCov workflow is run with:

```
hyb_results = CUQDyn1_Plus_HybridCov(cost_handle, dynamics_handle, nstates, n_params, ...
    guess_params, lb_params, ub_params, alp, ...
    times, all_state_data, ic, observed_data, observed_idx, hybDir, meigo_opts);
```

HybridCov uses the same observed-state conformal bands as the FIM workflow. For unobserved states, it keeps the FIM marginal parameter uncertainty scale but uses the leave-one-out parameter ensemble to estimate the parameter correlation structure.

HybridCov is an empirical covariance variant, not a distribution-free guarantee. In the shared-fit LV simulation-based calibration script, FIM and HybridCov gave similar predator coverage, so HybridCov should be checked per model rather than assumed to improve every example.

S2.1.5 Method 3: Bootstrap Trajectory UQ

The bootstrap workflow is a post-fit analysis. In the tutorial it starts from the FIM result:

```
boot = bootstrap_trajectory_uq(fim_results, dynamics_handle, cost_handle, ...
    lb_params, ub_params, bootDir, boot_opts);
```

The function estimates observation noise from the fitted observed-state residuals, simulates bootstrap observed datasets around the fitted trajectory, refits the parameters, propagates each fitted trajectory, and computes empirical quantile bands.

The bootstrap bands are latent ODE trajectory bands. They are not noisy observation prediction intervals. Therefore, coverage against noisy prey measurements is not the main target for the bootstrap output. The tutorial separates observed-data metrics from latent true-trajectory coverage for this reason.

S2.1.6 Output Files

Each tutorial run creates a timestamped folder such as:

```
EXAMPLES/LV/Tutorial_LV_three_UQ_methods_YYYY-MM-DD_HH-MM-SS/
```

The main subfolders are:

```
FIM/
HybridCov/
Bootstrap_from_FIM/
```

Each method folder contains trajectory summary tables. FIM and HybridCov also contain UQ diagnostic workbooks and `run_options.xlsx`, which records the effective optimizer, ODE, and UQ settings. The bootstrap folder contains `bootstrap_run_options.xlsx`. The tutorial root contains the cross-method comparison workbook:

```
three_uq_methods_metric_comparison.xlsx
```

S2.1.7 Interpreting a Representative LV Run

With `meigo_opts.maxeval = 1500` and `boot_opts.n_boot = 50`, the user observed:

Method	State	Latent coverage (%)	Mean interval width	Normalized mean interval width
FIM	Prey	100	14.967	0.19445
FIM	Predator	100	13.557	0.18131
HybridCov	Prey	100	14.970	0.19448
HybridCov	Predator	100	13.613	0.18207
Bootstrap	Prey	100	2.5075	0.032577
Bootstrap	Predator	100	10.715	0.14331

For this single LV dataset, the bootstrap latent trajectory bands were sharper than the delta-method bands while still covering the known latent trajectory. This is encouraging, but it is not a calibration guarantee. Repeated simulation studies are needed to assess whether the bootstrap bands maintain nominal coverage across datasets.

The same run also showed low bootstrap coverage against noisy prey observations. That is expected because the bootstrap output is a latent trajectory band. If the goal is coverage of future noisy observations, an observation-noise predictive interval should be added on top of the latent trajectory uncertainty.

S2.1.8 Generalizing to Other Examples

For new models, prefer starting from a high-level problem definition rather than writing legacy dynamics and cost files by hand. Copy `EXAMPLES/problem_definition_template.m`, fill in the states, parameters, ODE right-hand sides, observed states, bounds, and `problem.cost` residual scaling, plus `problem.synthetic_data` settings, then generate the legacy modules and a data-generation script:

```
problem = define_problem_MyModel();
cuqdyn_validate_problem(problem);
cuqdyn_generate_problem_files(problem, myExampleDir, 'Overwrite', true);
cuqdyn_generate_data_script(problem, fullfile(myExampleDir, 'generated_problem'), ...
    'Overwrite', true);
```

The maintained examples can be checked together from the repository root:

```
reports = check_generated_problem_definitions();
```

For a compact hidden-state stress test after the LV tutorial, use LinearCascade3. It is a three-state linear cascade where only the downstream state `x3` is observed, so the upstream states `x1` and `x2` are inferred indirectly:

```
cd EXAMPLES/LinearCascade
diagnose_LinearCascade3_known_truth
```

The script compares CUQDyn1_Plus against an independent matrix-exponential reference and writes `linear_cascade3_numerics_summary.xlsx`. The related `SBC_LinearCascade3_FIM_vs_HybridCov_sharedfit` scripts are useful for comparing FIM and HybridCov hidden-state coverage under repeated synthetic datasets.

The trajectory table function can be called after any CUQDyn1_Plus, CUQ-Dyn1_Plus_HybridCov, or bootstrap result:

```
state_names = {'State1', 'State2', 'State3'};  % replace with model names
metrics = save_trajectory_nrmse_tables(results, resultDir, state_names);
```

Use one name per state, in the same order as the ODE state vector and CSV data columns. For example:

```
% LV
state_names = {'Prey', 'Predator'};

% SIR
state_names = {'Susceptible', 'Infected', 'Recovered'};

% AP
```

```
state_names = {'AlphaPinene', 'Dipentene', 'AlloOcimene', ...
    'Pyronene', 'Dimer'};
```

For FIM and HybridCov results, diagnostics can also be added:

```
diagnose_uq_quality(results, resultDir, param_names, state_names, lb_params, ub_params);
```

For bootstrap trajectory UQ, first run one of the main CUQDyn1_Plus methods, then call `bootstrap_trajectory_uq` using the fitted result as input.

S2.1.9 Practical Recommendations

- Use FIM bands as the fastest baseline for prediction UQ.
- Use HybridCov as an empirical covariance alternative and check it with diagnostics or calibration studies.
- Use bootstrap trajectory UQ when local Gaussian approximations look fragile, especially when covariance diagnostics show high condition numbers or strong parameter correlations.
- Interpret bootstrap bands as latent trajectory uncertainty unless observation noise is explicitly added to form noisy-observation prediction intervals.
- For publication-quality bootstrap summaries, use more than the cheap tutorial settings and check successful replicate counts.

S3 Representative Figures From Example Results

The figures below are copied from the latest relevant result folders available in the local repository at generation time. They make this tutorial report self-contained enough to inspect the main workflows without opening each result directory manually.

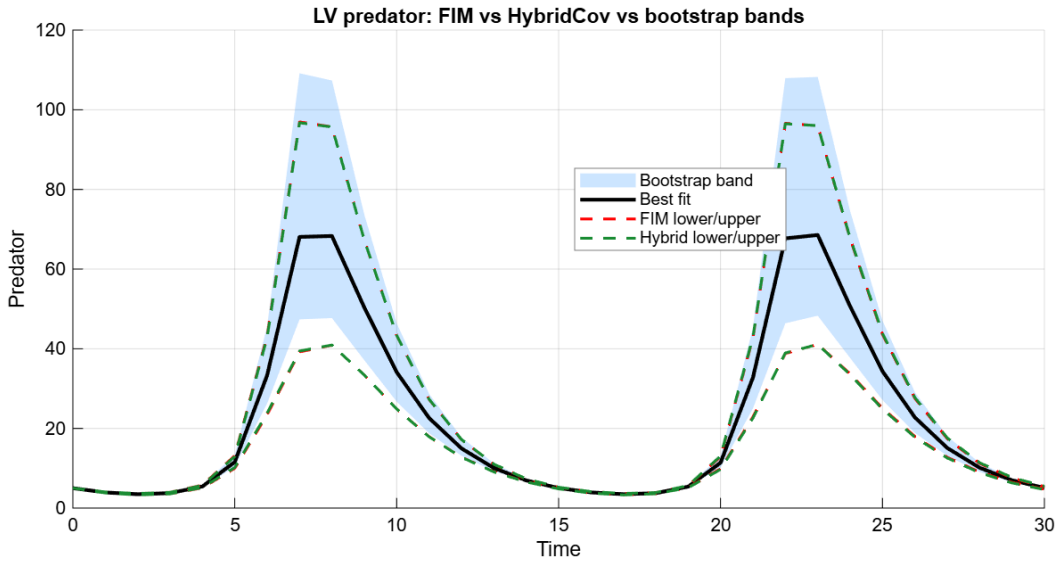

Figure S1: LV predator comparison across FIM, HybridCov, and bootstrap.

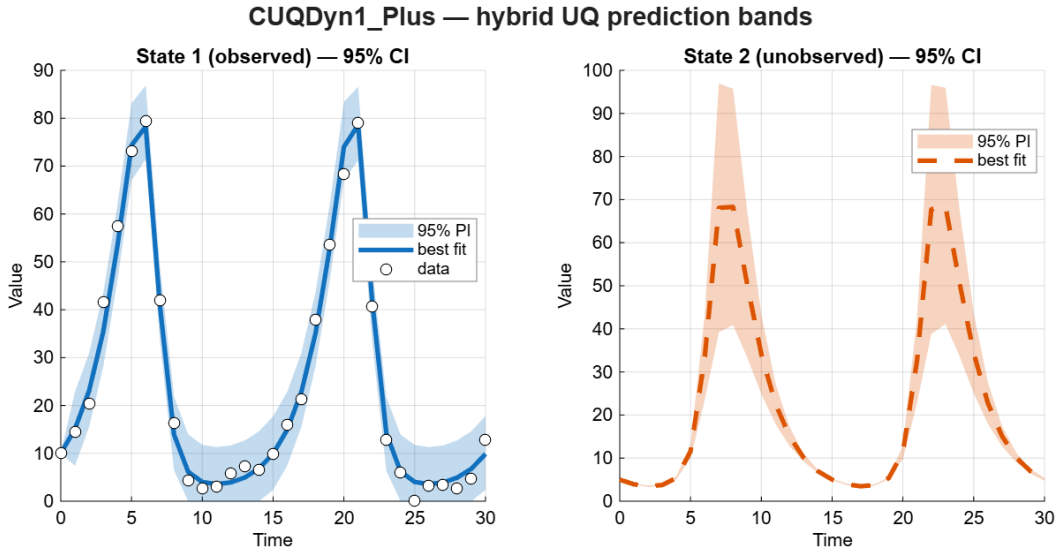

Figure S2: LV FIM example output.

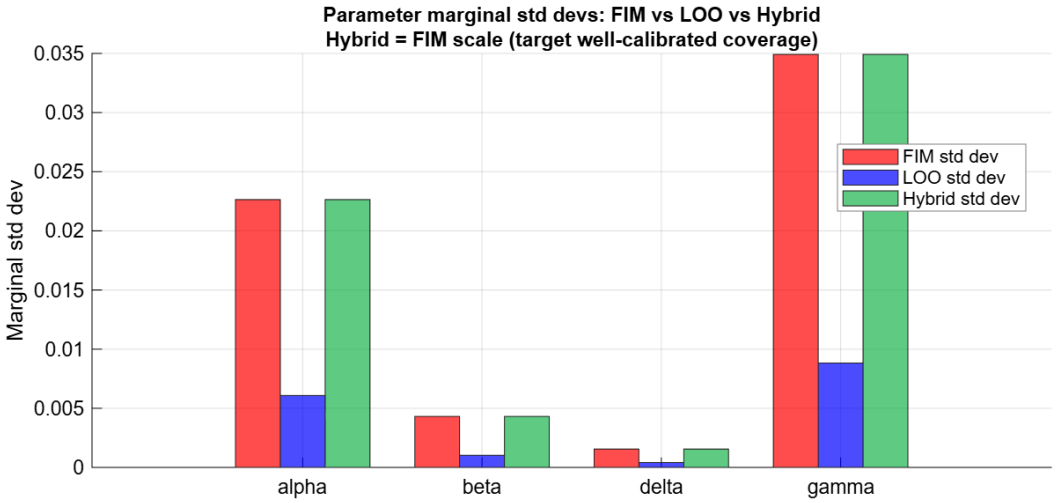

Figure S3: LV HybridCov marginal parameter standard deviation diagnostic.

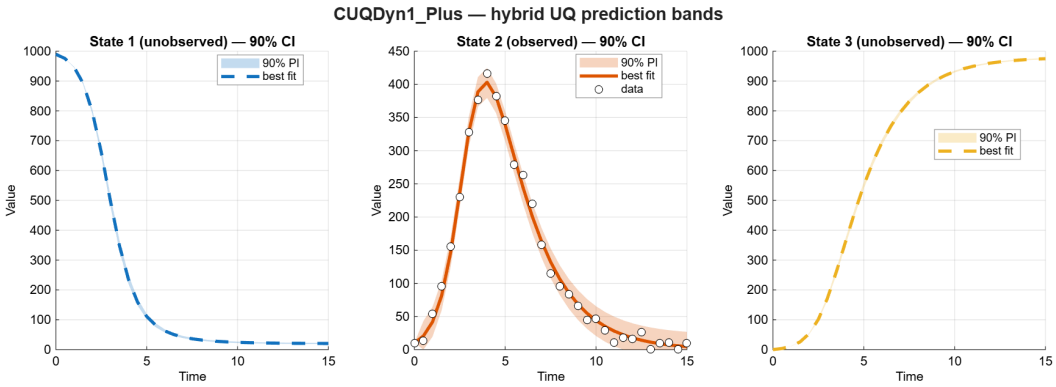

Figure S4: SIR FIM example output.

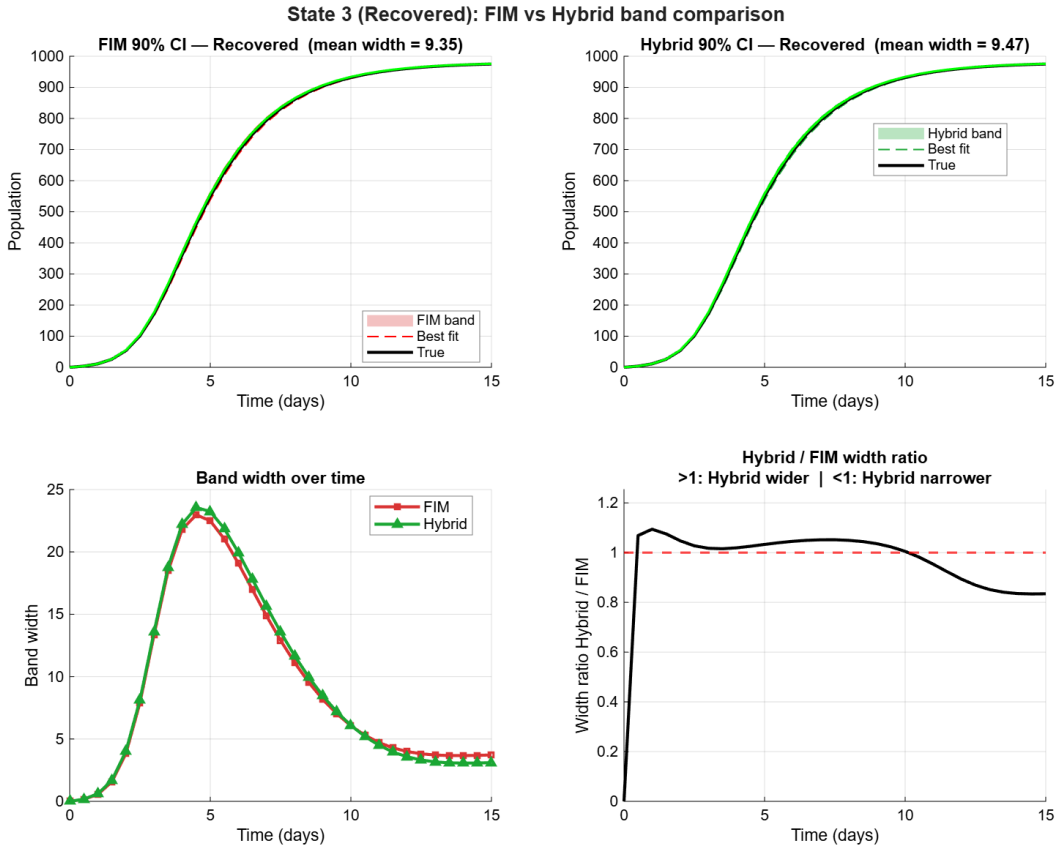

Figure S5: SIR HybridCov band comparison for the recovered state.

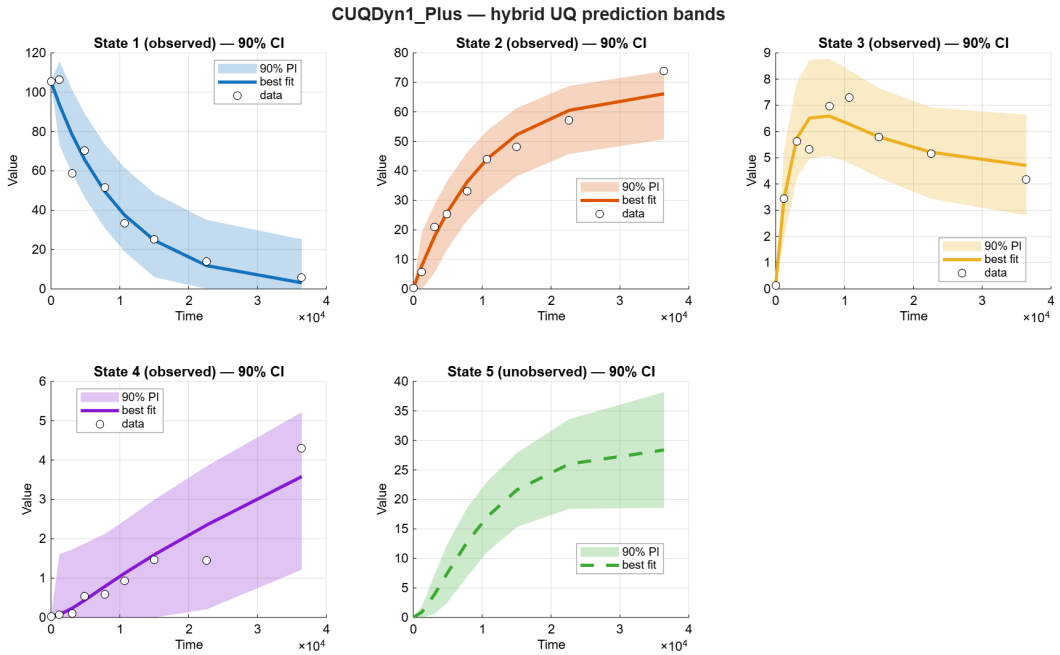

Figure S6: Alpha-pinene FIM example output.

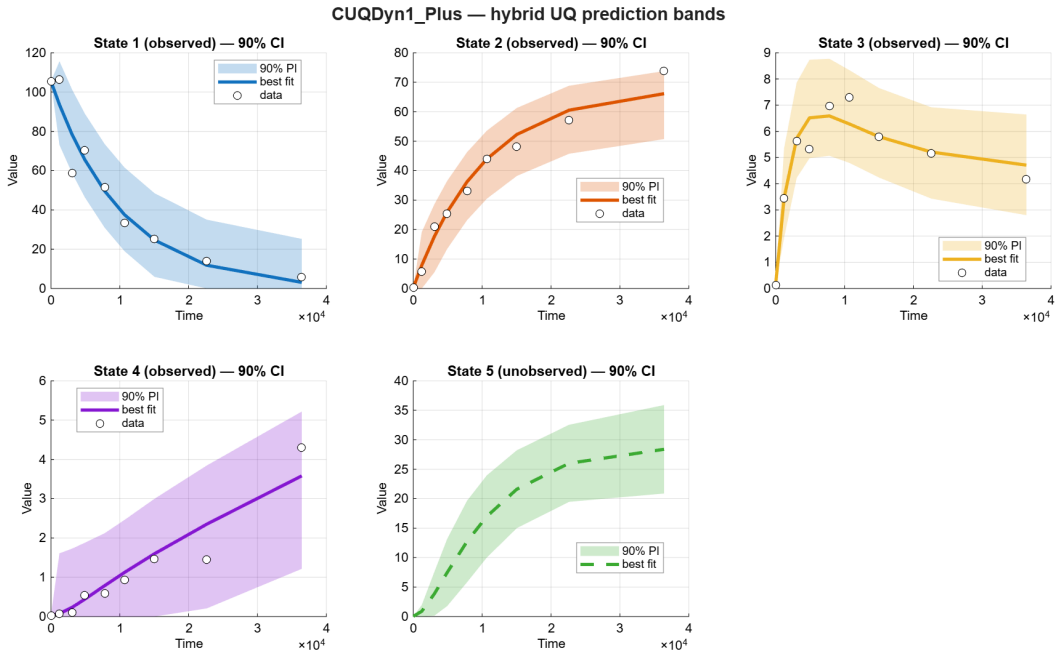

Figure S7: Alpha-pinene HybridCov example output.

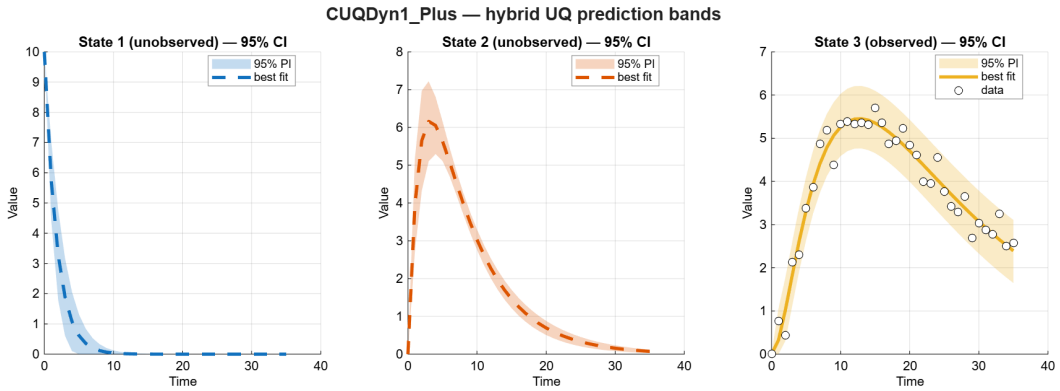

Figure S8: LinearCascade3 known-truth diagnostic output.

Part II. Empirical results across benchmark models

S4 Scope

This report was generated automatically from the latest available result folders in the local CUQ-Dyn1_Plus repository. It includes the maintained non-SBC examples, the latest LV three-method tutorial output, and the latest available SBC calibration folders. AP and NF-kB results use the latest folders available on 2026-07-20.

Table S1: Result folders used in this report.

Label	Source folder
LV FIM	`EXAMPLES/LV/Results_LV2_CUQDyn1_Plus_2026-07-17_07-39-44`
LV HybridCov	`EXAMPLES/LV/Results_LV2_HybridCov_2026-07-17_07-42-11`
LV tutorial	`EXAMPLES/LV/Tutorial_LV_three_UQ_methods_2026-07-17_08-20-08`
LV SBC	`EXAMPLES/LV/SBC_Results_LV2_FIM_vs_HybridCov_sharedfit_2026-07-17_08-32-07`
SIR FIM	`EXAMPLES/SIR/Results_SIR_CUQDyn1Plus_2026-07-17_07-44-13`
SIR HybridCov	`EXAMPLES/SIR/Results_SIR_HybridCov_2026-07-17_07-45-37`
SIR SBC	`EXAMPLES/SIR/SBC_Results_SIR_FIM_vs_HybridCov_2026-07-17_09-54-22`
AP FIM	`EXAMPLES/AP/Results_AP_partobs1_4_2026-07-17_07-47-19`
AP HybridCov	`EXAMPLES/AP/Results_AP_HybridCov_partobs1_4_2026-07-17_07-49-38`
AP SBC	`EXAMPLES/AP/SBC_Results_AP_FIM_vs_HybridCov_2026-07-17_11-45-09`
NFKB FIM	`EXAMPLES/NFKB/Results_NFKB_2026-07-17_07-53-10`
NFKB HybridCov	`EXAMPLES/NFKB/Results_NFKB_2026-07-17_08-04-42`
LinearCascade	`EXAMPLES/LinearCascade/Results_LinearCascade_known_truth_2026-07-17_08-16-47`
LinearCascade3	`EXAMPLES/LinearCascade/Results_LinearCascade3_known_truth_2026-07-17_08-18-17`
LinearCascade3 SBC	`EXAMPLES/LinearCascade/SBC_Results_LinearCascade3_sharedfit_2026-07-17_14-10-31`

S5 Trajectory Fit and Prediction-UQ Summaries

Table S2: Trajectory uncertainty summaries from trajectory_nrmse_tables.xlsx where available.

Example	Method	State	Coverage (%)	Mean width	Norm. width
LV	FIM	Prey	100	14.967	0.188
LV	FIM	Predator	100	0	–
LV	HybridCov	State1	100	14.967	0.188
LV	HybridCov	State2	100	0	–
SIR	FIM	State1	100	0	–
SIR	FIM	State2	100	43.48	0.104
SIR	FIM	State3	100	0	–
SIR	HybridCov	State1	100	0	–
SIR	HybridCov	State2	100	43.48	0.104
SIR	HybridCov	State3	100	0	–
AP	FIM	State1	100	37.841	0.376
AP	FIM	State2	100	20.509	0.279
AP	FIM	State3	100	3.201	0.447
AP	FIM	State4	100	2.949	0.69
AP	FIM	State5	100	0	–
AP	HybridCov	State1	100	37.841	0.376
AP	HybridCov	State2	100	20.509	0.279
AP	HybridCov	State3	100	3.2	0.447
AP	HybridCov	State4	100	2.949	0.69
AP	HybridCov	State5	100	0	–
LinearCascade	FIM	Hidden x1	100	0	–
LinearCascade	FIM	Observed x2	100	0.587	0.102
LinearCascade3	FIM	Hidden x1	100	0	–
LinearCascade3	FIM	Hidden x2	100	0	–
LinearCascade3	FIM	Observed x3	97.222	1.409	0.247

Table S3: Trajectory error summaries from trajectory_nrmse_tables.xlsx where available.

Example	Method	State	RMSE	MAE	NRMSE (%)
LV	FIM	Prey	2.325	1.751	2.927
LV	FIM	Predator	0	0	–
LV	HybridCov	State1	2.325	1.751	2.927
LV	HybridCov	State2	0	0	–
SIR	FIM	State1	0	0	–
SIR	FIM	State2	10.037	8.167	2.409
SIR	FIM	State3	0	0	–
SIR	HybridCov	State1	0	0	–
SIR	HybridCov	State2	10.037	8.167	2.409
SIR	HybridCov	State3	0	0	–
AP	FIM	State1	8.172	5.351	8.125
AP	FIM	State2	3.541	2.705	4.817
AP	FIM	State3	0.569	0.367	7.95
AP	FIM	State4	0.403	0.264	9.433
AP	FIM	State5	0	0	–
AP	HybridCov	State1	8.172	5.351	8.125
AP	HybridCov	State2	3.541	2.705	4.817
AP	HybridCov	State3	0.569	0.367	7.95
AP	HybridCov	State4	0.403	0.264	9.433
AP	HybridCov	State5	0	0	–
LinearCascade	FIM	Hidden x1	0	0	–
LinearCascade	FIM	Observed x2	0.134	0.104	2.319
LinearCascade3	FIM	Hidden x1	0	0	–
LinearCascade3	FIM	Hidden x2	0	0	–
LinearCascade3	FIM	Observed x3	0.305	0.237	5.357

S6 LV Three-Method Tutorial

Method	State	Latent coverage (%)	Mean width	Norm. width
FIM	Prey	100	14.968	0.194
FIM	Predator	100	13.347	0.179
HybridCov	Prey	100	14.968	0.194
HybridCov	Predator	100	13.402	0.179
Bootstrap	Prey	100	2.444	0.032
Bootstrap	Predator	100	14.216	0.19

Table S4: Latent true-trajectory coverage from the LV three-method tutorial.

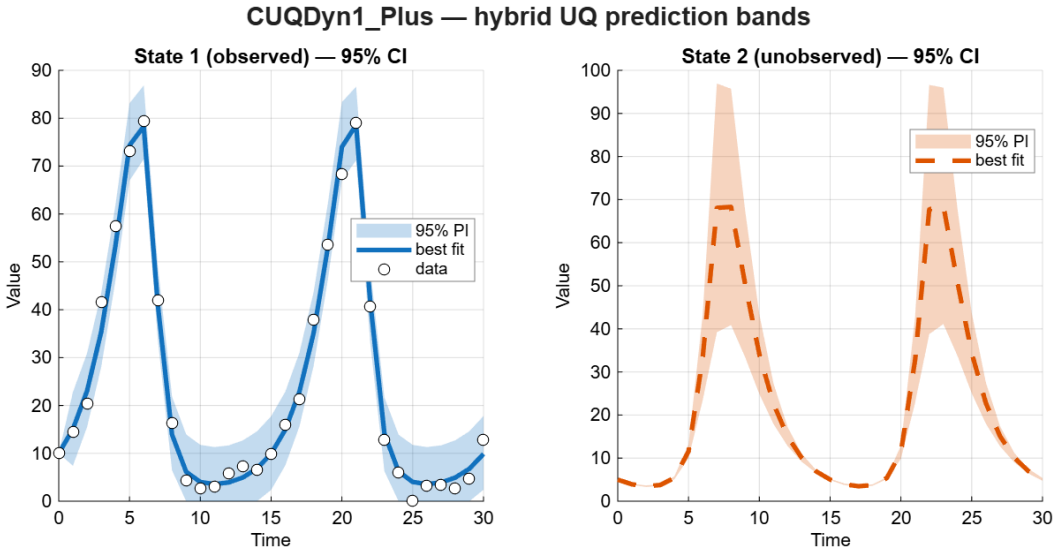

Figure S9: LV FIM fit and UQ bands.

S7 Linear Cascade Known-Truth Checks

Example	Cov. rel. Fro. error	Hidden std rel. error	Max traj. ref. error
LinearCascade	6.221e-08	1.306e-08	2.987e-08
LinearCascade3	1.405e-06	1.354e-06	8.890e-07

Table S5: Known-truth numerical reference checks for LinearCascade examples.

S8 Simulation-Based Calibration

SBC tables report hidden-state or unobserved-state coverage from the latest available calibration folders. Nominal coverage differs by script: the LV and LinearCascade3 shared-fit runs report 95% nominal intervals, while the SIR and AP logs report 90% nominal intervals.

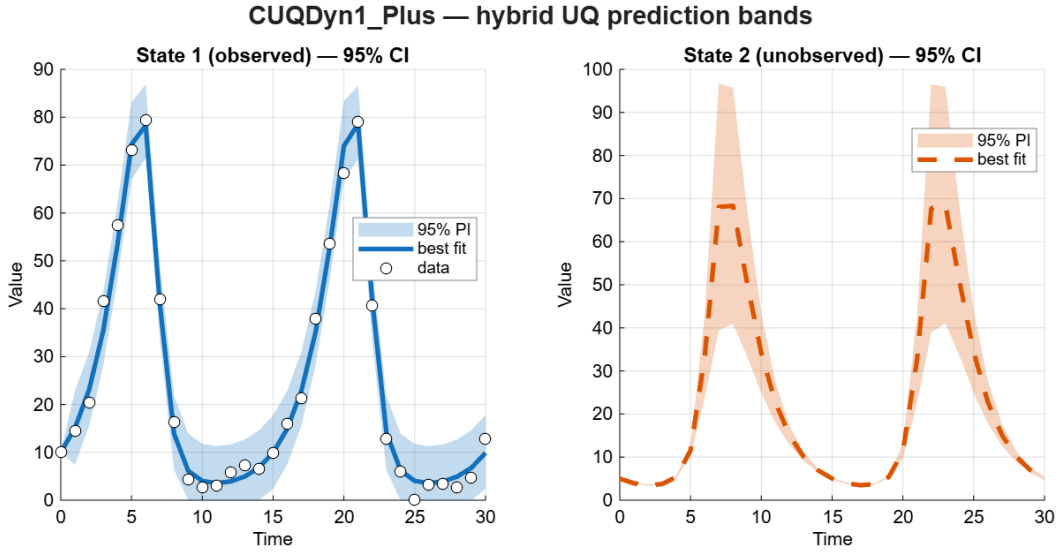

Figure S10: LV HybridCov fit and UQ bands.

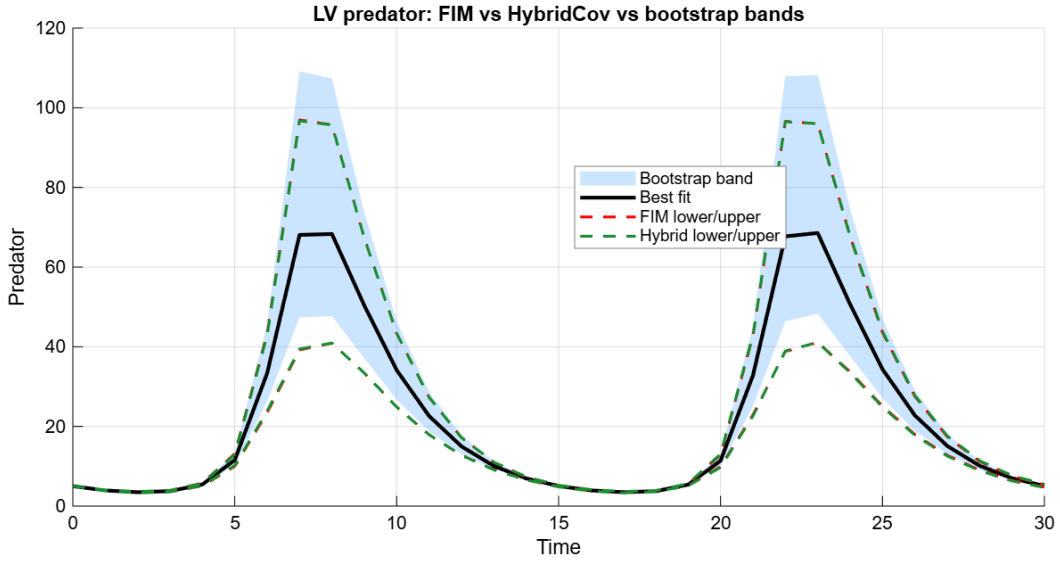

Figure S11: LV predator comparison across FIM, HybridCov, and bootstrap.

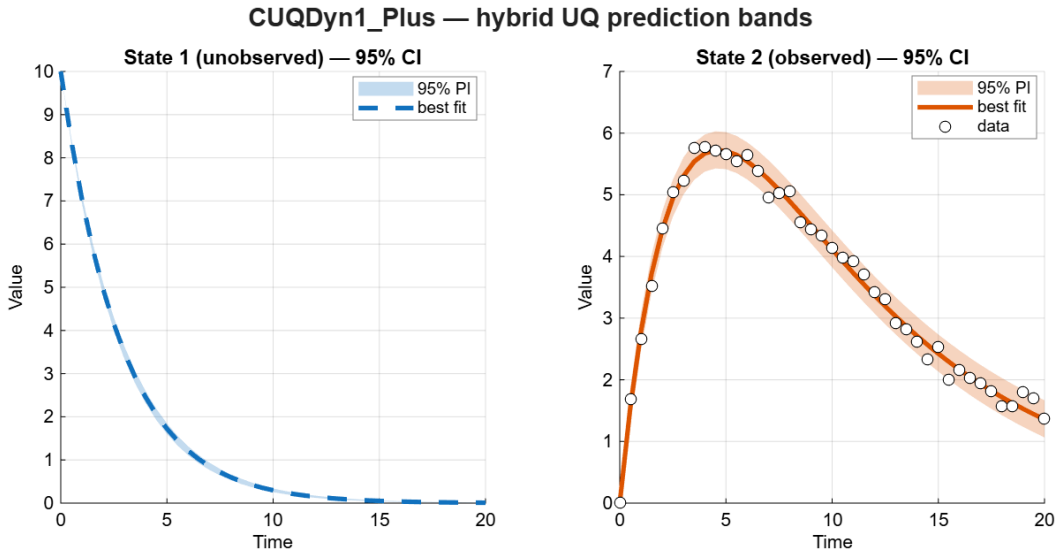

Figure S12: Two-state LinearCascade known-truth diagnostic.

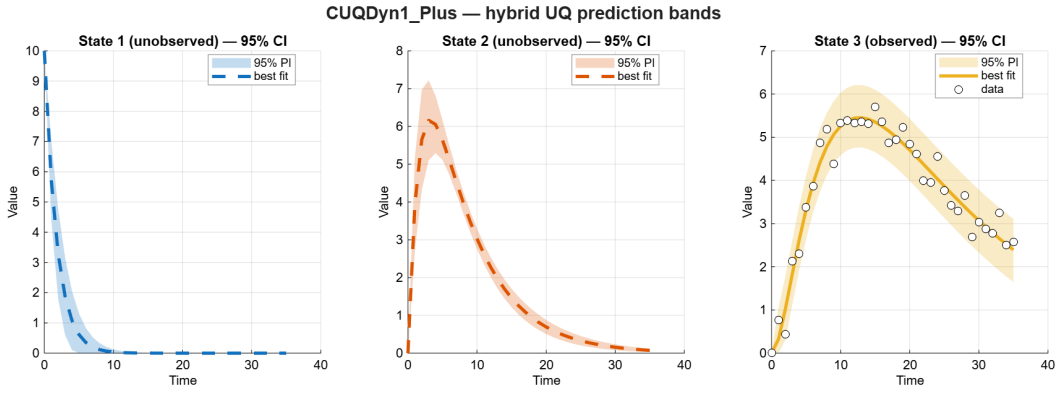

Figure S13: Three-state LinearCascade3 known-truth diagnostic.

Table S6: SBC coverage and band-width summaries. The nominal column gives each example's target two-sided level (LV and LinearCascade3 at 95%; SIR and AP at 90%), since coverage should be read against it.

Example	Method	State	Nominal (%)	Mean ptwise cov. (%)	Std ptwise cov. (%)	Simult. cov. (%)	Mean width
LV	FIM	Predator	95	93.732	17.416	74	16.12
LV	HybridCov	Predator	95	93.804	18.018	72	16.582
SIR	FIM	Susceptible	90	85	32.7	80	10.098
SIR	FIM	Recovered	90	86.6	29.6	72	9.287
SIR	HybridCov	Susceptible	90	87.5	27	78	9.991
SIR	HybridCov	Recovered	90	87.2	29.8	78	9.252
AP	FIM	State 5	90	88.5	25.5	78	4.015
AP	HybridCov	State 5	90	87.2	28.2	80	3.958
LinearCascade3	FIM	Hidden x1	95	73.829	41.017	68	0.67
LinearCascade3	HybridCov	Hidden x1	95	73.829	41.017	68	0.67
LinearCascade3	FIM	Hidden x2	95	74.629	42.636	68	0.762
LinearCascade3	HybridCov	Hidden x2	95	73.2	43.246	68	0.751

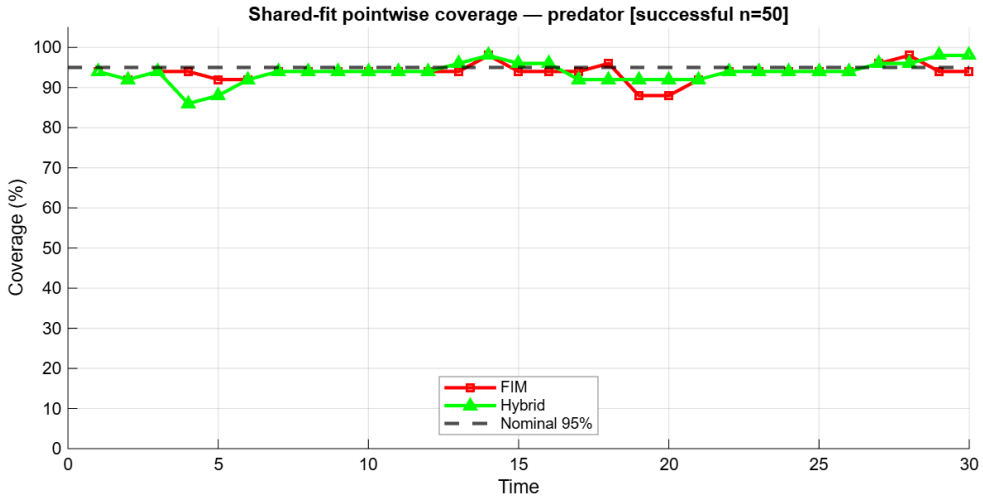

Figure S14: LV shared-fit SBC coverage by time.

S9 Example Figures

S10 Notes and Caveats

- Tables are extracted from generated Excel workbooks and SBC logs; no examples were rerun by this report script.
- NF-kB result workbooks do not currently include trajectory_nrmse_tables.xlsx, so NF-kB contributes figures and result folders but not a compact trajectory summary table.
- Bootstrap intervals in the LV tutorial are latent trajectory intervals, not noisy-observation prediction intervals.
- The LV shared-fit SBC folder contains figures and a log, but no xlsx summary; its SBC row is computed from per-replicate coverage and width lines in the log.

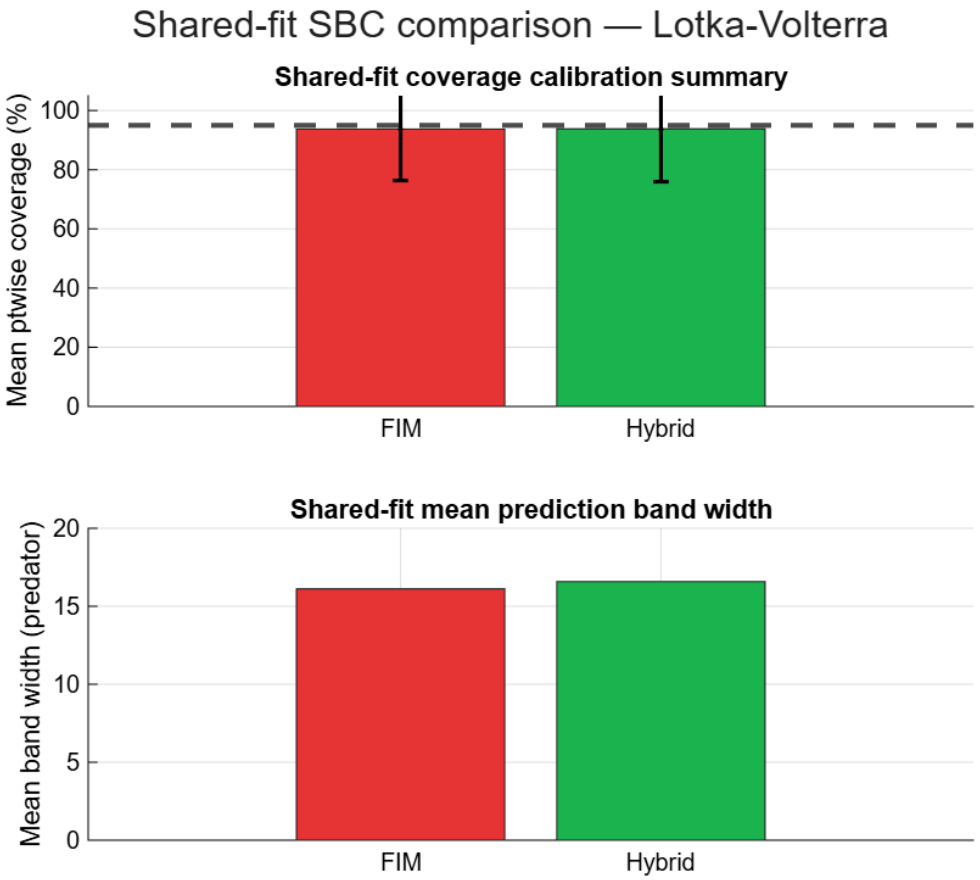

Figure S15: LV shared-fit SBC summary.

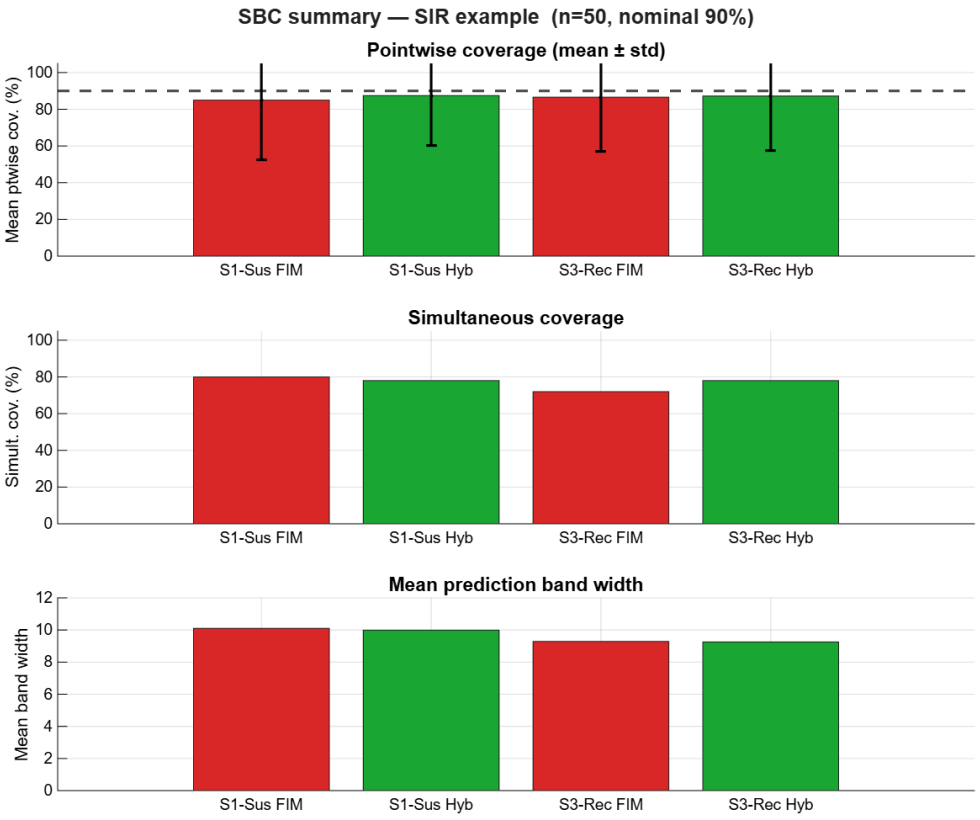

Figure S16: SIR SBC summary.

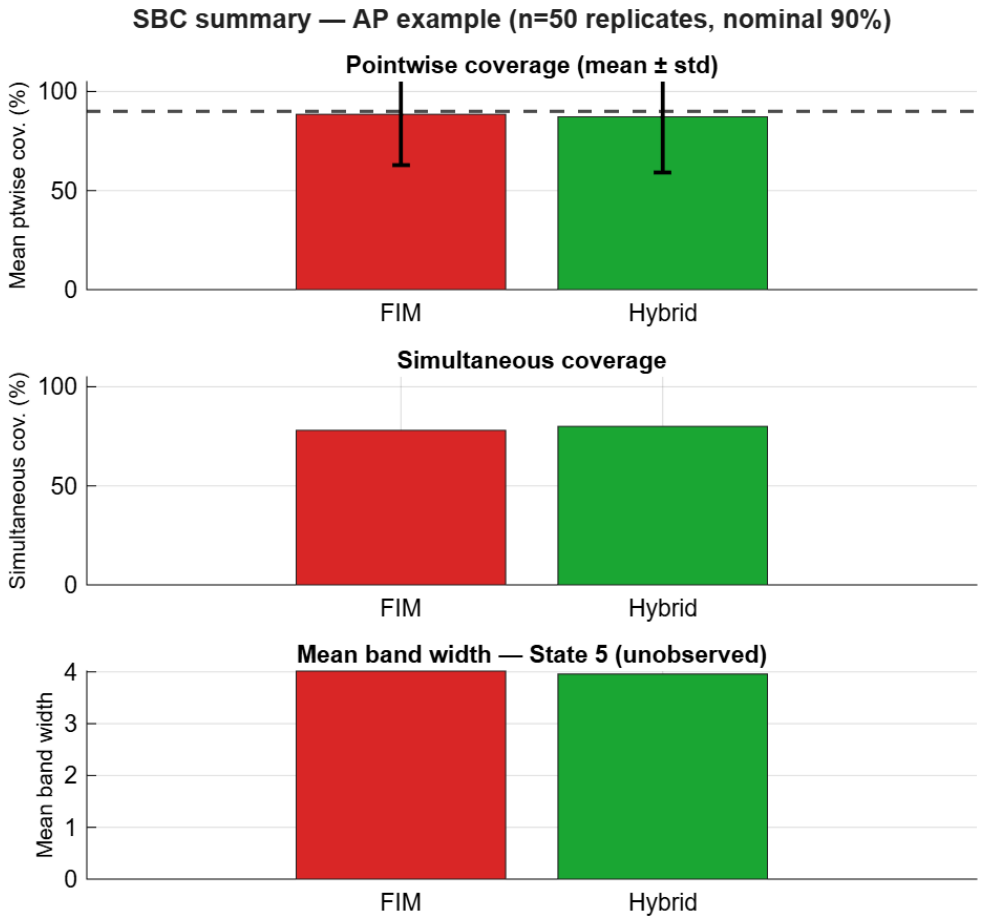

Figure S17: AP SBC summary.

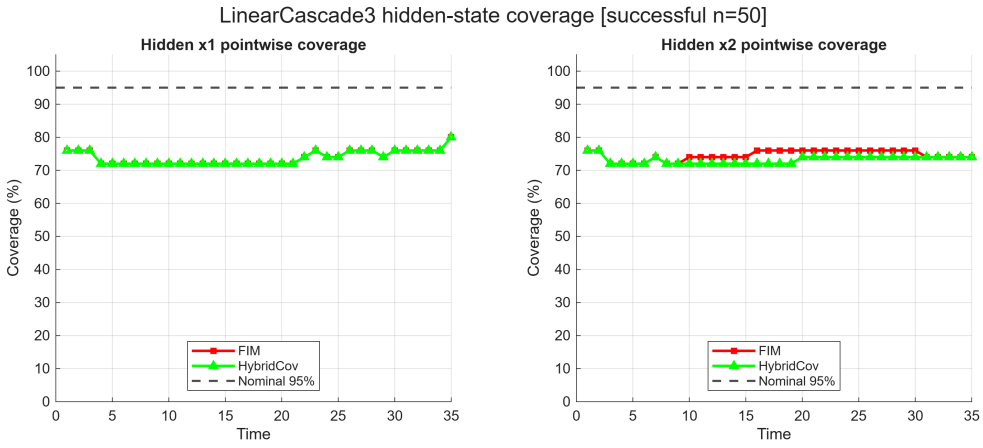

Figure S18: LinearCascade3 shared-fit SBC hidden-state coverage by time.

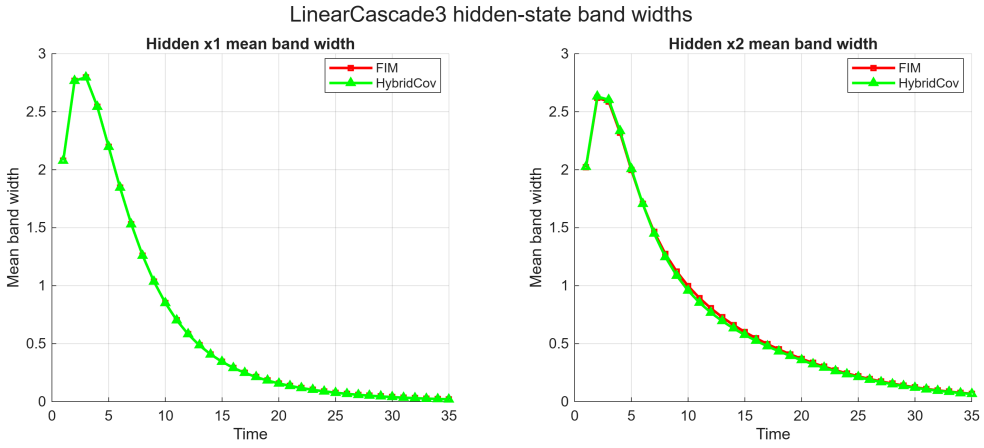

Figure S19: LinearCascade3 shared-fit SBC hidden-state band widths.

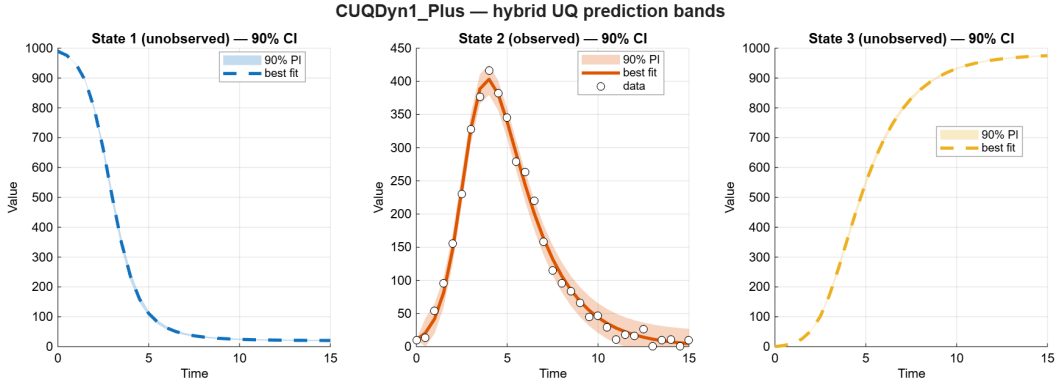

Figure S20: SIR FIM fit and UQ bands.

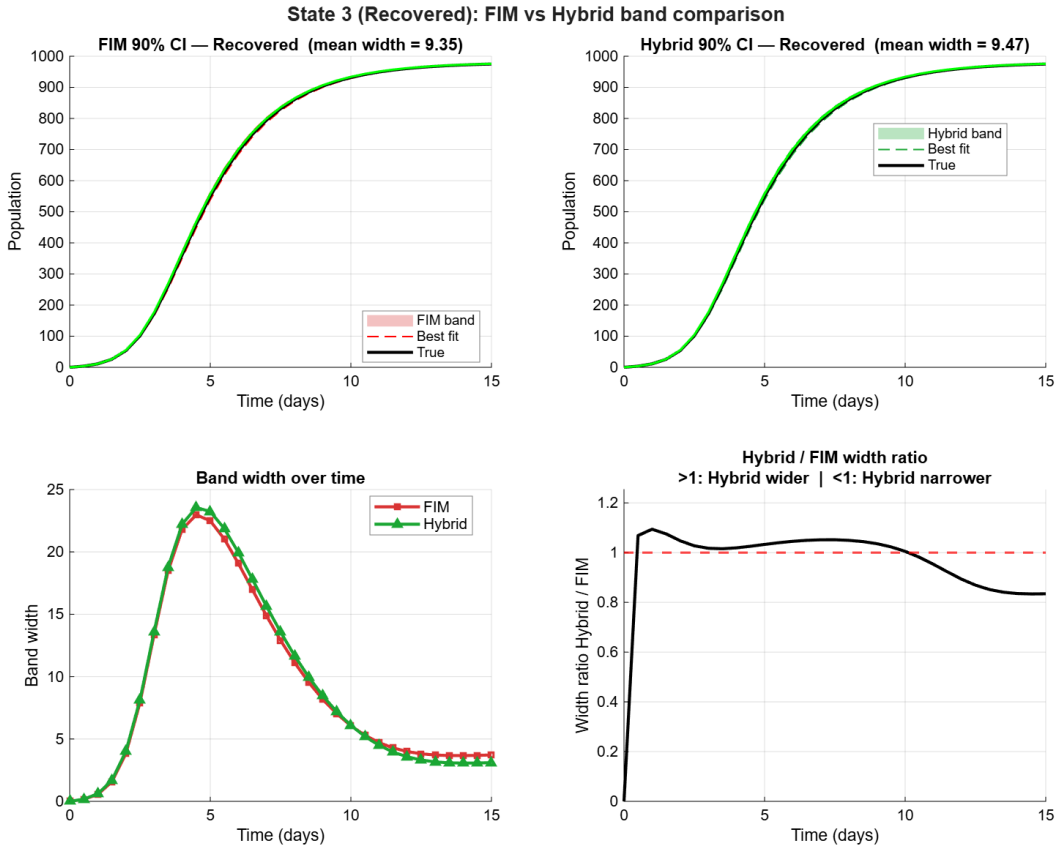

Figure S21: SIR HybridCov diagnostic band comparison for recovered state.

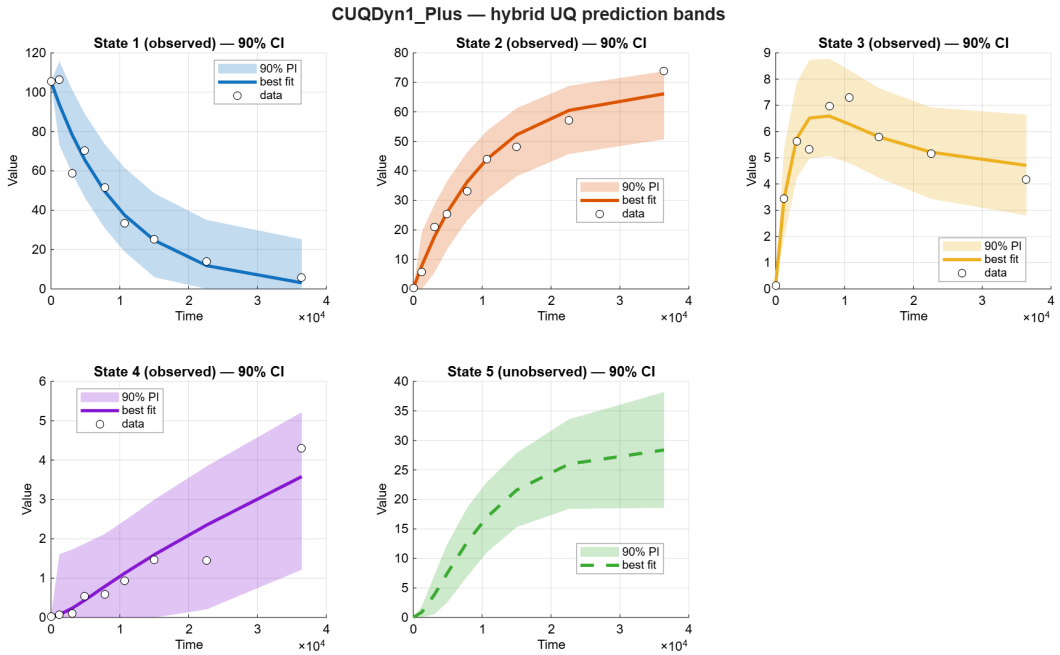

Figure S22: AP FIM fit and UQ bands after widened bounds.

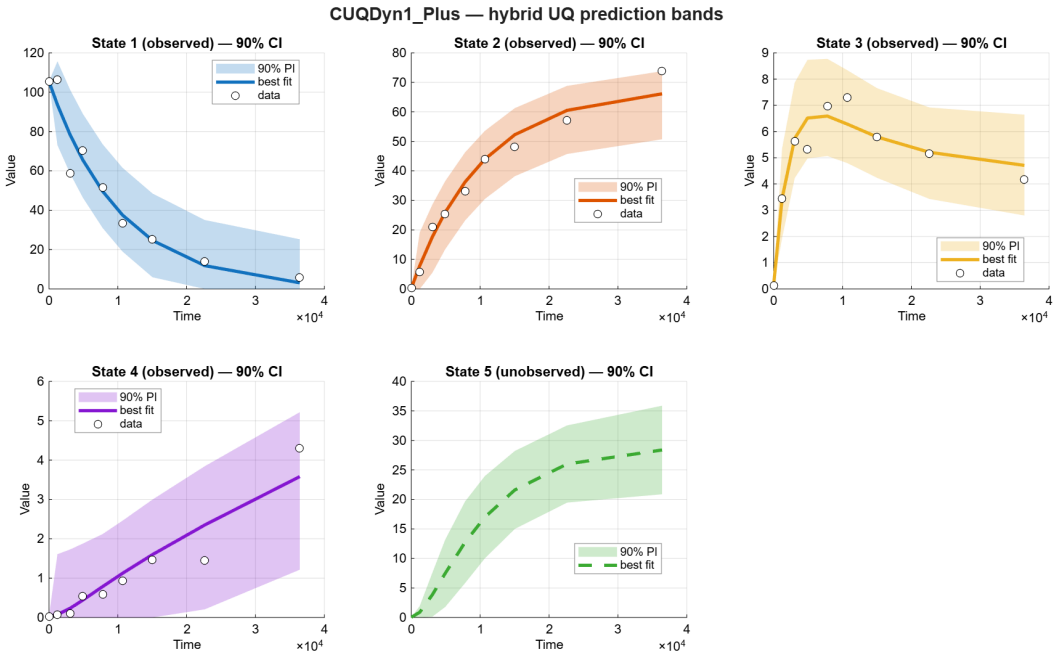

Figure S23: AP HybridCov fit and UQ bands after widened bounds.

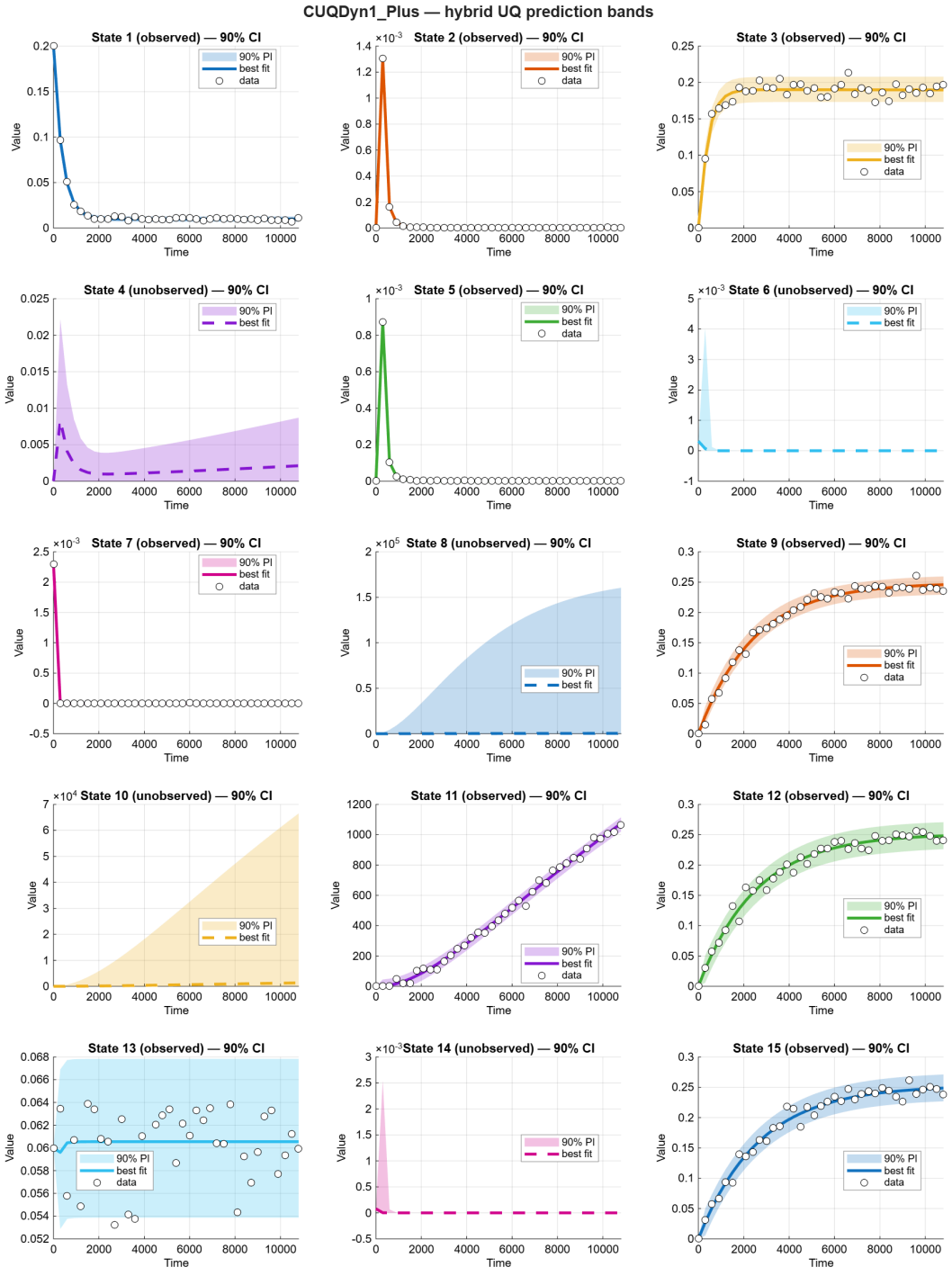

Figure S24: NF-kB FIM fit and UQ bands after widened bounds.

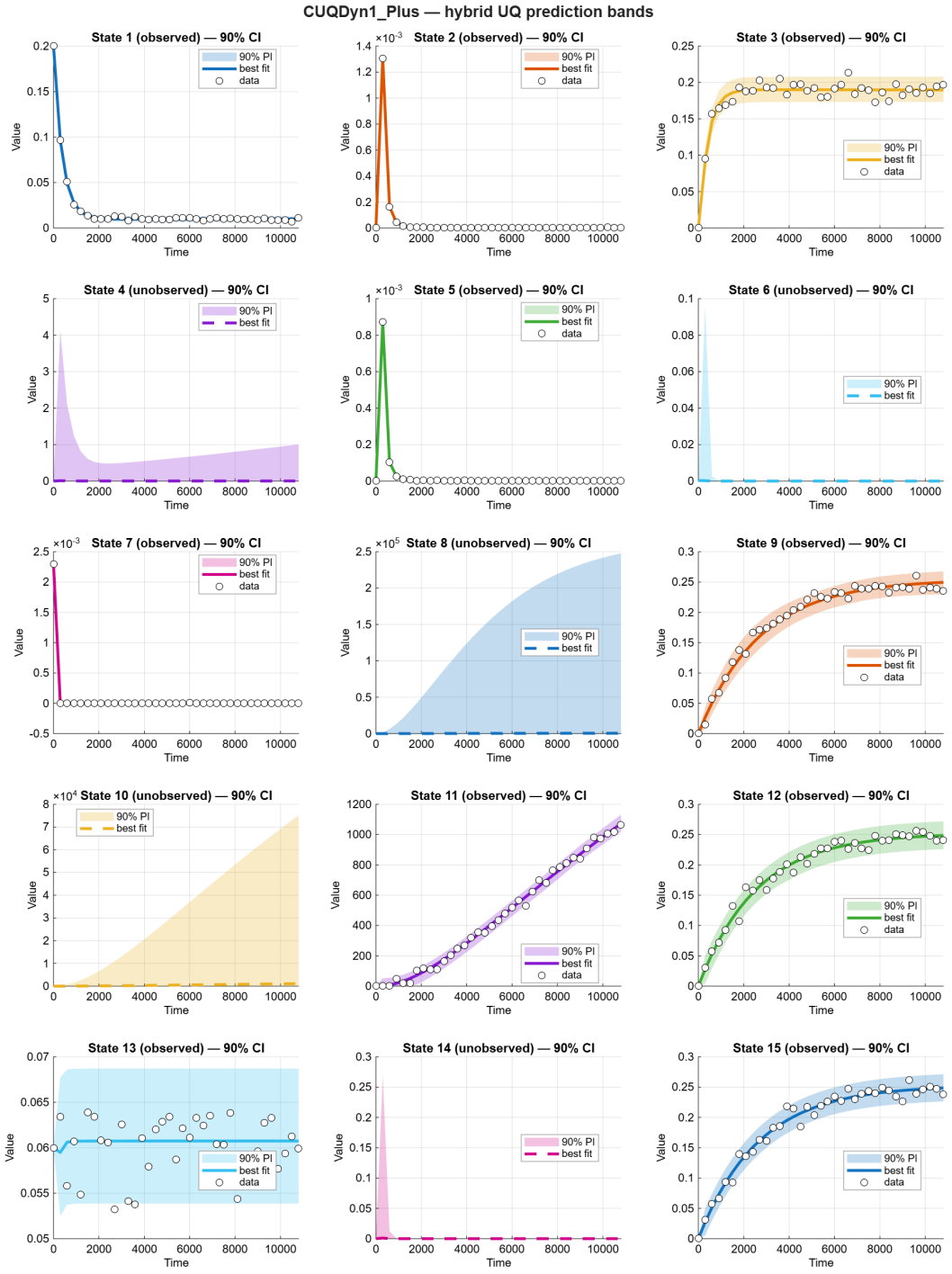

Figure S25: NF-kB HybridCov fit and UQ bands after widened bounds.

Part III. CUQDyn1_Plus versus PyMC ABC-SMC

S11 Scope and Provenance

This report compares the latest completed CUQDyn1_Plus and PyMC ABC-SMC outputs for the maintained Bayesian comparison examples: LV, SIR, AP, and NF-kB. The tables are generated from the shared comparison CSV files produced by `pymc_matlab/compare_cuqdyn_pymc.m`. The PyMC workflows read the same data CSV files used by the corresponding CUQDyn examples.

Comparison source folder: `pymc_matlab\results\comparison\2026-07-17_15-38-40`.

S12 Example Settings

Model	Data CSV	States	Observed states	Noise/data model	True/nominal parameters	Notes
LV	EXAMPLES/LV/data/lv2_synthetic_data_noi10_partobs_1.csv	prey, predator	prey	10% synthetic observation noise	alpha=0.5, beta=0.02, delta=0.02, gamma=0.5	Predator is hidden after t=0; all methods use the same CSV.
SIR	EXAMPLES/SIR/data/sir_data.csv	susceptible, infected, recovered	infected	10% proportional synthetic noise on infected state	beta=0.002, gamma=0.5	Susceptible and recovered are hidden after t=0.
AP	EXAMPLES/AP/data/AP_measurementData_1_4.csv	y1, y2, y3, y4, y5	y1-y4	alpha-pinene measurement dataset used by CUQDyn examples	p1=5.93e-05, p2=2.96e-05, p3=2.05e-05, p4=2.75e-04, p5=4.00e-05	y5 is hidden after t=0; AP residuals/ PyMC ABC distance normalize observed states by scale.
NFKB	EXAMPLES/NFKB/data/NFKB_synthetic_data_5n_36st_partobs10.csv	15-state NF-kB signaling model	y1, y2, y3, y5, y7, y9, y11, y12, y13, y15	5% synthetic noise, 36 sampling times	29 kinetic parameters p1-p29; nominal values from run_bayes_nfkb.m	Five states are hidden: y4, y6, y8, y10, y14.

S13 Method Settings

S13.1 CUQDyn1_Plus Settings

Method	Summary	Optimizer/refits	Uncertainty setting
CUQDyn1_Plus FIM	best-fit trajectory with FIM/delta-method parameter covariance propagated to prediction bands	MEIGO/eSS global fit with lsqnonlin local solver, example-specific maxeval from run script	nominal bands with per-example `uq.alp` (0.025/95% for LV and LinearCascade3; 0.05/90% for SIR, AP, NF-kB)
CUQDyn1_Plus HybridCov	same CUQDyn fit workflow, replacing FIM correlations with LOO-refit correlations while retaining FIM marginal scale	MEIGO/eSS global fit with LOO refits according to example options	nominal HybridCov bands with the same per-example `uq.alp` as FIM

For each model and method, the latest timestamped result folder containing the expected CUQDyn result MAT file was used:

Model	Method	Result folder
LV	CUQDyn1_Plus FIM	EXAMPLES\LV\Results_LV2_CUQDyn1_Plus_2026-07-17_07-39-44
LV	CUQDyn1_Plus HybridCov	EXAMPLES\LV\Results_LV2_HybridCov_2026-07-17_07-42-11
SIR	CUQDyn1_Plus FIM	EXAMPLES\SIR\Results_SIR_CUQDyn1Plus_2026-07-17_07-44-13

Model	Method	Result folder
SIR	CUQDyn1_Plus HybridCov	EXAMPLES\SIR\Results_SIR_HybridCov_2026-07-17_07-45-37
AP	CUQDyn1_Plus FIM	EXAMPLES\AP\Results_AP_partobs1_4_2026-07-17_07-47-19
AP	CUQDyn1_Plus HybridCov	EXAMPLES\AP\Results_AP_HybridCov_partobs1_4_2026-07-17_07-49-38
NFKB	CUQDyn1_Plus FIM	EXAMPLES\NFKB\Results_NFKB_2026-07-17_07-53-10
NFKB	CUQDyn1_Plus HybridCov	EXAMPLES\NFKB\Results_NFKB_2026-07-17_08-04-42
LV	PyMC ABC-SMC	pymc_matlab/results/lv
SIR	PyMC ABC-SMC	pymc_matlab/results/sir
AP	PyMC ABC-SMC	pymc_matlab/results/ap
NFKB	PyMC ABC-SMC	pymc_matlab/results/nfkb

S13.2 PyMC ABC-SMC Settings

Model	Priors	Distance/normalization	epsilon	draws/chains	SMC thresholds	Trajectory ensemble
LV	Uniform: alpha [0.10,1.00], beta/delta [0.004,0.04], gamma [0.10,1.00]	PyMC Simulator ABC distance on observed prey	3	4000 x 4 chains	threshold=0.6, correlation_threshold=0.01	500 posterior draws; observed-state export includes RMSE noise augmentation
SIR	Uniform: beta [1e-4,1e-2], gamma [0.01,2.0]	PyMC Simulator ABC distance on infected state	20	1000 x 4 chains	threshold=0.3, correlation_threshold=0.1	500 posterior draws; observed-state export includes RMSE noise augmentation
AP	Uniform bounds equal nominal parameter value times [0.05,5.0], matching current CUQDyn AP runs	ABC distance on y1-y4 after dividing each state by its time mean	0.5	1000 x 4 chains	threshold=0.5, correlation_threshold=0.1	500 posterior draws; observed-state export includes RMSE noise augmentation

Model	Priors	Distance/normalization	epsilon	draws/chains	SMC thresholds	Trajectory ensemble
NFKB	Uniform bounds equal nominal parameter value times [0.1,4.0], matching current CUQDyn NF-kB support	custom normalized L1 distance over observed states, each scaled by observed maximum	1.0	1000 x 4 chains	threshold=0.3, correlation_threshold=0.1; 1000-draw comparison run is flagged if R-hat ¿ 1.01 or ESS ¡ 100 x chains	500 posterior draws; observed-state export includes RMSE noise augmentation; independent 4000-draw rerun passes the gate marginally but remains prior-dominated

PyMC trajectory bands in this report are quantiles of the exported posterior trajectory ensembles. Observed-state PyMC bands include an additive observation-noise term estimated from posterior-mean RMSE; hidden-state PyMC bands remain latent parameter-uncertainty bands.

S14 High-Level Parameter Comparison

Model	Method	n params	Mean rel. err. %	Median rel. err. %	Params ¡=20% rel. err.
AP	CUQDyn1_Plus FIM	5	23.0	16.4	3/5
AP	CUQDyn1_Plus HybridCov	5	23.0	16.4	3/5
AP	PyMC ABC-SMC	5	97.0	49.9	2/5
LV	CUQDyn1_Plus FIM	4	4.8	3.3	4/4
LV	CUQDyn1_Plus HybridCov	4	4.8	3.3	4/4
LV	PyMC ABC-SMC	4	6.2	3.6	4/4
NFKB	CUQDyn1_Plus FIM	29	45.2	26.1	14/29
NFKB	CUQDyn1_Plus HybridCov	29	54.7	48.0	10/29
NFKB	PyMC ABC-SMC	29	107.4	104.3	0/29
SIR	CUQDyn1_Plus FIM	2	0.7	0.9	2/2
SIR	CUQDyn1_Plus HybridCov	2	0.7	0.9	2/2
SIR	PyMC ABC-SMC	2	0.6	0.9	2/2

S15 High-Level Trajectory-Band Comparison

Model	Method	Obs. states	Obs. cov. (%)	Obs. RMSE	Obs. width	Hidden states	Hidden width
AP	CUQDyn1_Plus FIM	4	100.0	3.17	16.1	1	9.93
AP	CUQDyn1_Plus HybridCov	4	100.0	3.17	16.1	1	10
AP	PyMC ABC-SMC	4	97.2	3.62	18.7	1	20.4
LV	CUQDyn1_Plus FIM	1	100.0	2.33	15	1	13.3
LV	CUQDyn1_Plus HybridCov	1	100.0	2.33	15	1	13.4
LV	PyMC ABC-SMC	1	96.8	2.58	10.9	1	15.7
NFKB	CUQDyn1_Plus FIM	10	96.2	2.15	8.68	5	4.993e+04
NFKB	CUQDyn1_Plus HybridCov	10	96.8	2.16	10.1	5	7.151e+04
NFKB	PyMC ABC-SMC	10	98.1	19	88.4	5	442
SIR	CUQDyn1_Plus FIM	1	100.0	10	43.5	2	9.45
SIR	CUQDyn1_Plus HybridCov	1	100.0	10	43.5	2	8.9
SIR	PyMC ABC-SMC	1	100.0	10	43.6	2	17.6

S16 Side-by-Side Predictive Figures

The following figures place PyMC, CUQDyn FIM, and CUQDyn HybridCov panels side by side for each state. Within a given state row, all method panels use the same y-axis limits. Black markers are observations where available.

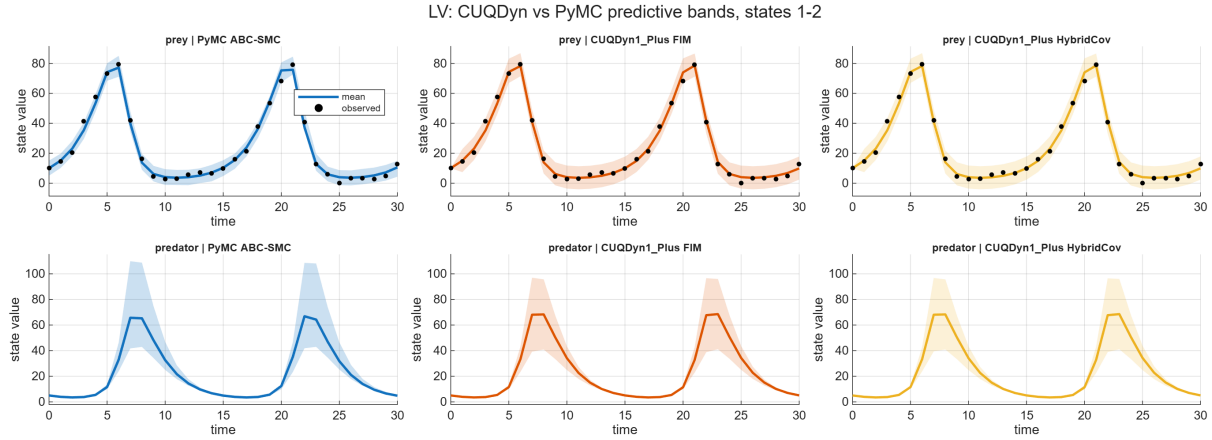

Figure S26: LV side-by-side CUQDyn1_Plus versus PyMC predictive comparison.

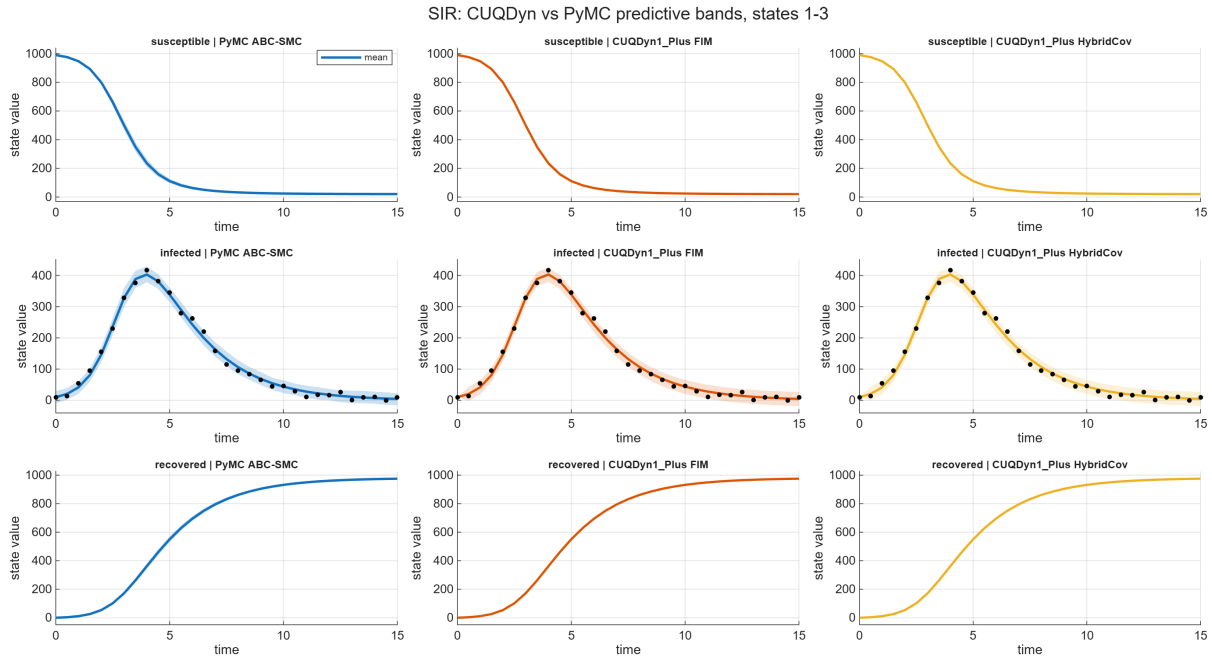

Figure S27: SIR side-by-side CUQDyn1_Plus versus PyMC predictive comparison.

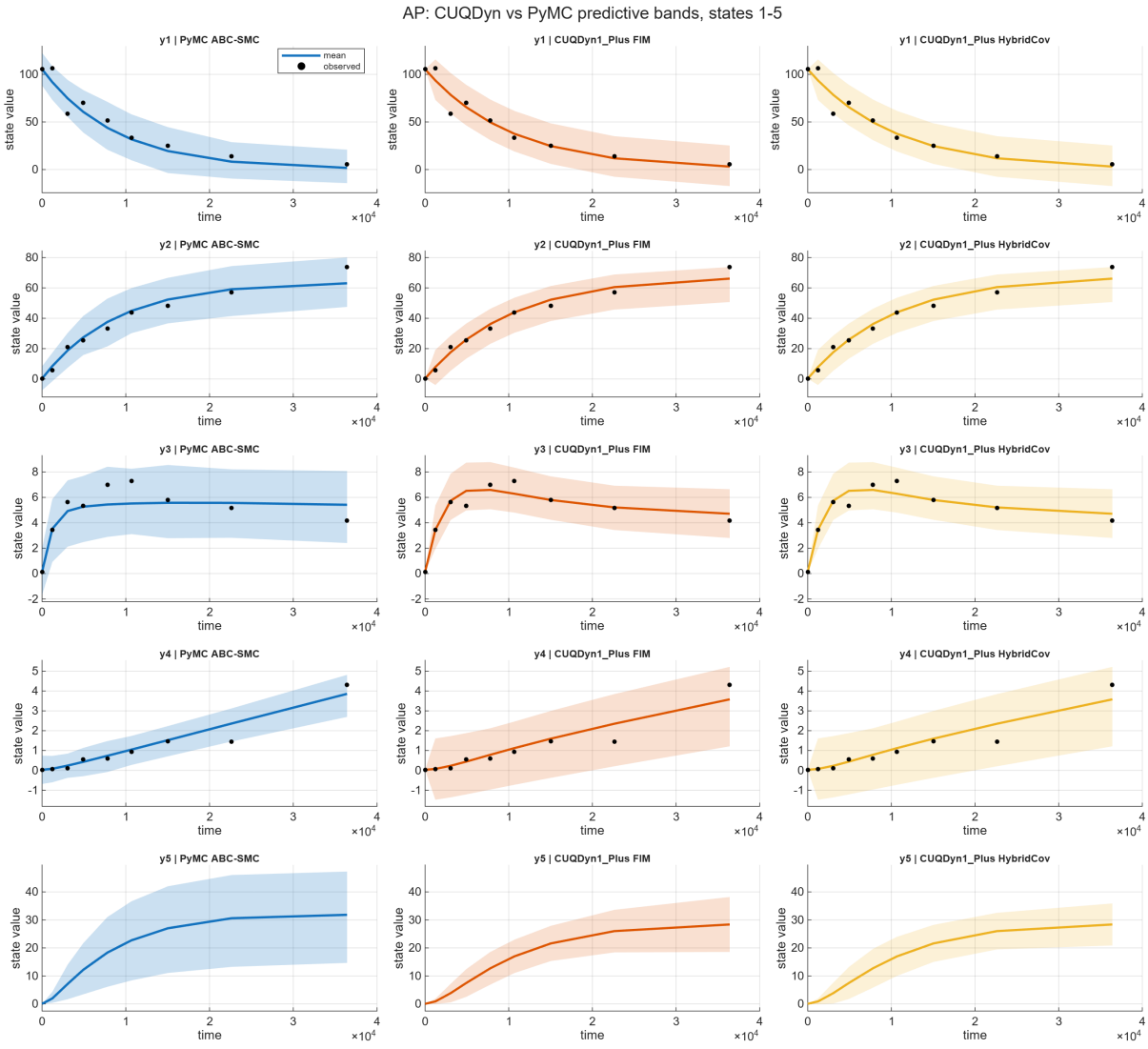

Figure S28: AP side-by-side CUQDyn1_Plus versus PyMC predictive comparison.

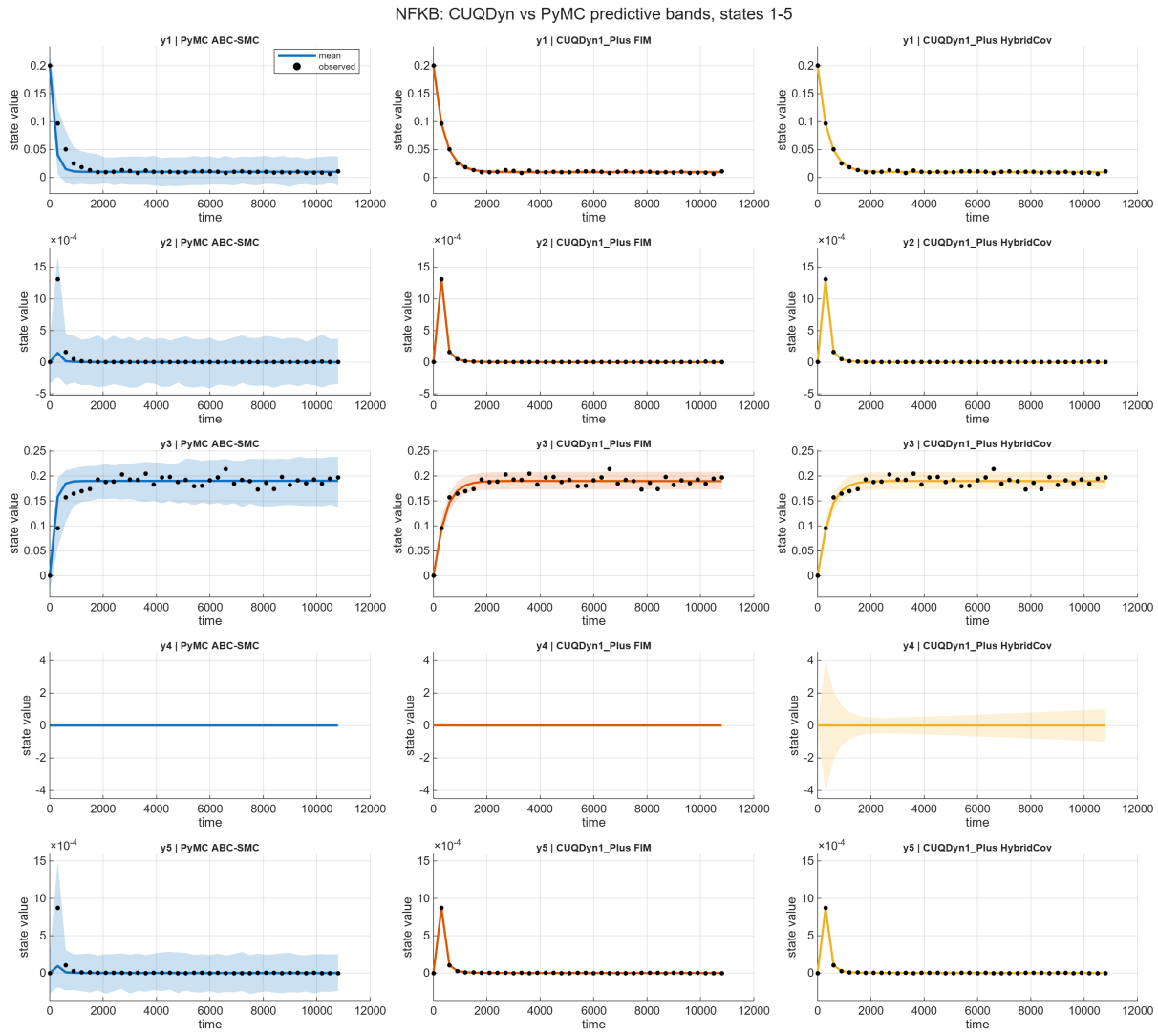

Figure S29: NFKB states 1-5 side-by-side CUQDyn1_Plus versus PyMC predictive comparison.

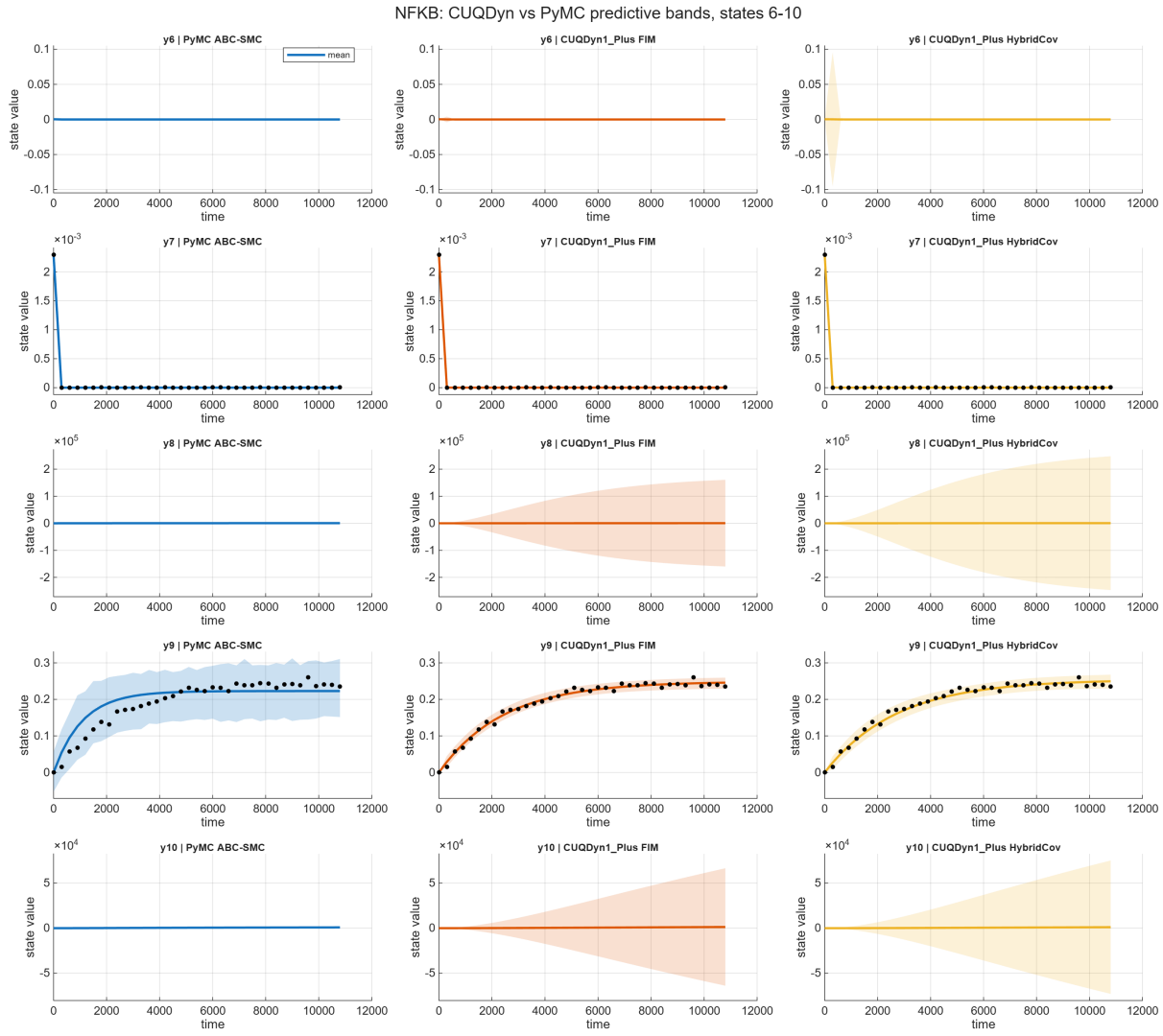

Figure S30: NFKB states 6-10 side-by-side CUQDyn1_Plus versus PyMC predictive comparison.

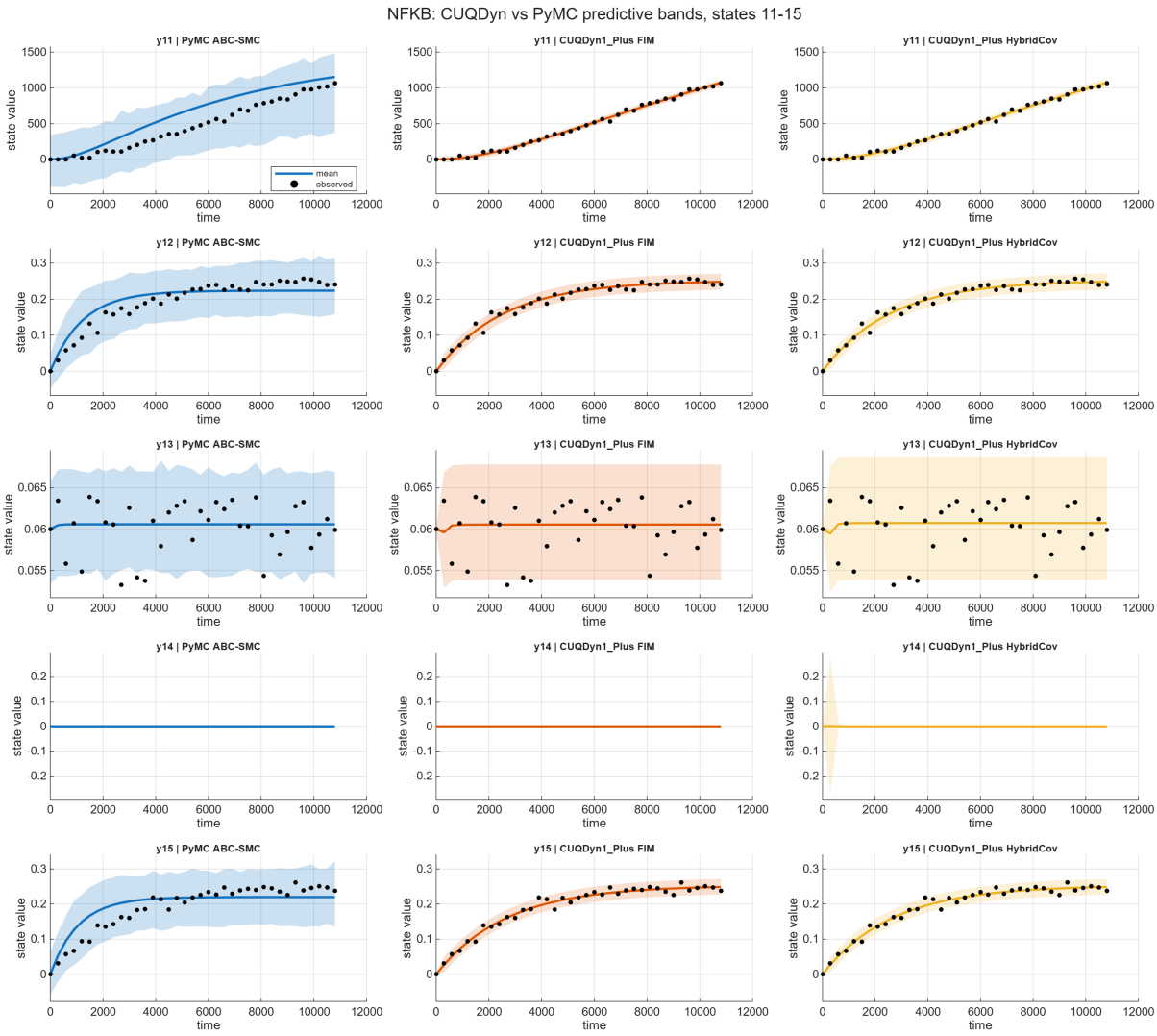

Figure S31: NFKB states 11-15 side-by-side CUQDyn1_Plus versus PyMC predictive comparison.

S17 Full Parameter Table

S17.1 LV

Method	Parameter	True	Estimate	StdDev	Lower	Upper	RelErr %
PyMC ABC-SMC	alpha	0.5	0.511	0.028	0.462	0.572	2.3
PyMC ABC-SMC	beta	0.02	0.0231	0.00533	0.0141	0.0347	15.7
PyMC ABC-SMC	delta	0.02	0.0194	0.00187	0.016	0.0233	3.1
PyMC ABC-SMC	gamma	0.5	0.482	0.0423	0.404	0.568	3.6
CUQDyn1_Plus FIM	alpha	0.5	0.507	0.0226	0.463	0.552	1.5
CUQDyn1_Plus FIM	beta	0.02	0.0223	0.00431	0.0138	0.0307	11.4
CUQDyn1_Plus FIM	delta	0.02	0.0194	0.00155	0.0164	0.0225	2.9
CUQDyn1_Plus FIM	gamma	0.5	0.483	0.0349	0.415	0.552	3.3
CUQDyn1_Plus HybridCov	alpha	0.5	0.507	0.0226	0.463	0.552	1.5
CUQDyn1_Plus HybridCov	beta	0.02	0.0223	0.00431	0.0138	0.0307	11.4
CUQDyn1_Plus HybridCov	delta	0.02	0.0194	0.00155	0.0164	0.0225	2.9
CUQDyn1_Plus HybridCov	gamma	0.5	0.483	0.0349	0.415	0.552	3.3

S17.2 SIR

Method	Parameter	True	Estimate	StdDev	Lower	Upper	RelErr %
PyMC ABC-SMC	beta	0.002	0.00198	2.537e-05	0.00193	0.00203	0.9
PyMC ABC-SMC	gamma	0.5	0.499	0.0104	0.479	0.519	0.3
CUQDyn1_Plus FIM	beta	0.002	0.00198	1.683e-05	0.00195	0.00201	0.9
CUQDyn1_Plus FIM	gamma	0.5	0.498	0.00683	0.485	0.512	0.4
CUQDyn1_Plus HybridCov	beta	0.002	0.00198	1.683e-05	0.00195	0.00201	0.9
CUQDyn1_Plus HybridCov	gamma	0.5	0.498	0.00683	0.485	0.512	0.4

S17.3 AP

Method	Parameter	True	Estimate	StdDev	Lower	Upper	RelErr %
PyMC ABC-SMC	p1	5.930e-05	6.811e-05	1.527e-05	4.265e-05	1.030e-04	14.9
PyMC ABC-SMC	p2	2.960e-05	4.437e-05	1.598e-05	1.844e-05	8.037e-05	49.9
PyMC ABC-SMC	p3	2.050e-05	1.975e-05	4.129e-06	1.310e-05	2.924e-05	3.6
PyMC ABC-SMC	p4	2.750e-04	7.802e-04	2.979e-04	2.258e-04	0.00132	183.7
PyMC ABC-SMC	p5	4.000e-05	1.332e-04	3.995e-05	4.927e-05	1.936e-04	233.1
CUQDyn1_Plus FIM	p1	5.930e-05	6.223e-05	3.306e-06	5.575e-05	6.871e-05	4.9

Method	Parameter	True	Estimate	StdDev	Lower	Upper	RelErr %
CUQDyn1_Plus FIM	p2	2.960e-05	3.446e-05	4.031e-06	2.656e-05	4.236e-05	16.4
CUQDyn1_Plus FIM	p3	2.050e-05	1.811e-05	1.923e-05	-1.958e-05	5.580e-05	11.7
CUQDyn1_Plus FIM	p4	2.750e-04	3.672e-04	2.940e-04	-2.091e-04	9.435e-04	33.5
CUQDyn1_Plus FIM	p5	4.000e-05	5.935e-05	9.009e-05	-1.172e-04	2.359e-04	48.4
CUQDyn1_Plus HybridCov	p1	5.930e-05	6.223e-05	3.306e-06	5.575e-05	6.871e-05	4.9
CUQDyn1_Plus HybridCov	p2	2.960e-05	3.446e-05	4.031e-06	2.656e-05	4.236e-05	16.4
CUQDyn1_Plus HybridCov	p3	2.050e-05	1.811e-05	1.923e-05	-1.958e-05	5.580e-05	11.7
CUQDyn1_Plus HybridCov	p4	2.750e-04	3.672e-04	2.940e-04	-2.091e-04	9.435e-04	33.5
CUQDyn1_Plus HybridCov	p5	4.000e-05	5.935e-05	9.009e-05	-1.172e-04	2.359e-04	48.4

S17.4 NFKB

Method	Parameter	True	Estimate	StdDev	Lower	Upper	RelErr %
PyMC ABC-SMC	p1	0.5	1.06	0.549	0.128	1.93	111.0
PyMC ABC-SMC	p2	0.2	0.415	0.213	0.0656	0.772	107.5
PyMC ABC-SMC	p3	0.1	0.211	0.106	0.031	0.38	111.0
PyMC ABC-SMC	p4	1	2.02	1.04	0.29	3.79	101.8
PyMC ABC-SMC	p5	0.1	0.203	0.104	0.0332	0.383	103.4
PyMC ABC-SMC	p6	5.000e-07	1.005e-06	5.302e-07	1.337e-07	1.902e-06	101.0
PyMC ABC-SMC	p7	1.000e-04	1.827e-04	8.114e-05	6.364e-05	3.469e-04	82.7
PyMC ABC-SMC	p8	4.000e-04	8.171e-04	3.884e-04	2.106e-04	0.00152	104.3
PyMC ABC-SMC	p9	0.5	0.915	0.445	0.208	1.76	82.9
PyMC ABC-SMC	p10	1.000e-04	1.898e-04	1.046e-04	2.606e-05	3.753e-04	89.8
PyMC ABC-SMC	p11	2.000e-05	4.008e-05	2.157e-05	5.147e-06	7.680e-05	100.4
PyMC ABC-SMC	p12	5.000e-07	9.982e-07	5.361e-07	1.345e-07	1.917e-06	99.6
PyMC ABC-SMC	p13	1.000e-04	2.065e-04	8.165e-05	7.238e-05	3.532e-04	106.5
PyMC ABC-SMC	p14	4.000e-04	9.269e-04	3.751e-04	2.703e-04	0.00153	131.7
PyMC ABC-SMC	p15	0.5	1.09	0.524	0.185	1.92	117.2
PyMC ABC-SMC	p16	3.000e-04	5.879e-04	3.170e-04	8.248e-05	0.00114	96.0
PyMC ABC-SMC	p17	0.0025	0.00582	0.00207	0.00203	0.00937	132.9
PyMC ABC-SMC	p18	0.1	0.192	0.102	0.031	0.377	92.5
PyMC ABC-SMC	p19	0.0015	0.00296	0.00162	3.367e-04	0.00573	97.5
PyMC ABC-SMC	p20	2.500e-05	6.291e-05	2.431e-05	1.181e-05	9.664e-05	151.6
PyMC ABC-SMC	p21	1.250e-04	3.130e-04	1.167e-04	6.529e-05	4.769e-04	150.4
PyMC ABC-SMC	p22	5	8.92	5.65	0.941	19	78.3
PyMC ABC-SMC	p23	0.0025	0.00497	0.00272	5.713e-04	0.00958	98.9
PyMC ABC-SMC	p24	0.01	0.0205	0.0107	0.00283	0.038	105.1
PyMC ABC-SMC	p25	0.001	0.00164	9.024e-04	3.156e-04	0.00351	63.7
PyMC ABC-SMC	p26	5.000e-04	0.0012	4.856e-04	2.390e-04	0.00193	139.8
PyMC ABC-SMC	p27	5.000e-07	1.047e-06	5.220e-07	1.455e-07	1.914e-06	109.3
PyMC ABC-SMC	p28	1.000e-04	2.091e-04	7.877e-05	7.554e-05	3.510e-04	109.1

Method	Parameter	True	Estimate	StdDev	Lower	Upper	RelErr %
PyMC ABC-SMC	p29	4.000e-04	9.505e-04	3.709e-04	2.711e-04	0.00154	137.6
CUQDyn1_Plus FIM	p1	0.5	0.365	6.74	-12.8	13.6	26.9
CUQDyn1_Plus FIM	p2	0.2	0.172	4.5	-8.66	9	14.0
CUQDyn1_Plus FIM	p3	0.1	0.0868	1.6	-3.04	3.21	13.2
CUQDyn1_Plus FIM	p4	1	0.887	0.797	-0.675	2.45	11.3
CUQDyn1_Plus FIM	p5	0.1	0.0894	0.0751	-0.0577	0.237	10.6
CUQDyn1_Plus FIM	p6	5.000e-07	1.884e-06	9.431e-04	-0.00185	0.00185	276.7
CUQDyn1_Plus FIM	p7	1.000e-04	9.937e-05	3.467e-06	9.257e-05	1.062e-04	0.6
CUQDyn1_Plus FIM	p8	4.000e-04	3.948e-04	1.763e-05	3.602e-04	4.293e-04	1.3
CUQDyn1_Plus FIM	p9	0.5	0.826	23	-44.3	46	65.1
CUQDyn1_Plus FIM	p10	1.000e-04	4.860e-05	1.080e-04	-1.632e-04	2.604e-04	51.4
CUQDyn1_Plus FIM	p11	2.000e-05	2.001e-06	0.001	-0.00197	0.00197	90.0
CUQDyn1_Plus FIM	p12	5.000e-07	5.006e-08	2.511e-05	-4.916e-05	4.926e-05	90.0
CUQDyn1_Plus FIM	p13	1.000e-04	1.025e-04	3.621e-06	9.543e-05	1.096e-04	2.5
CUQDyn1_Plus FIM	p14	4.000e-04	4.123e-04	1.838e-05	3.763e-04	4.483e-04	3.1
CUQDyn1_Plus FIM	p15	0.5	0.631	167	-327	328	26.1
CUQDyn1_Plus FIM	p16	3.000e-04	3.982e-04	2.156e-04	-2.430e-05	8.207e-04	32.7
CUQDyn1_Plus FIM	p17	0.0025	0.0025	2.312e-05	0.00245	0.00254	0.2
CUQDyn1_Plus FIM	p18	0.1	0.0819	21.7	-42.5	42.7	18.1
CUQDyn1_Plus FIM	p19	0.0015	2.666e-04	0.0978	-0.191	0.192	82.2
CUQDyn1_Plus FIM	p20	2.500e-05	2.585e-05	5.610e-07	2.475e-05	2.695e-05	3.4
CUQDyn1_Plus FIM	p21	1.250e-04	1.289e-04	3.550e-06	1.219e-04	1.359e-04	3.1
CUQDyn1_Plus FIM	p22	5	10.1	22	-32.9	53.2	102.1
CUQDyn1_Plus FIM	p23	0.0025	8.255e-04	0.0393	-0.0761	0.0778	67.0
CUQDyn1_Plus FIM	p24	0.01	0.0235	11.8	-23	23.1	134.6
CUQDyn1_Plus FIM	p25	0.001	6.502e-04	0.0191	-0.0369	0.0382	35.0
CUQDyn1_Plus FIM	p26	5.000e-04	7.854e-04	0.0022	-0.00352	0.00509	57.1
CUQDyn1_Plus FIM	p27	5.000e-07	8.453e-08	4.240e-05	-8.301e-05	8.318e-05	83.1
CUQDyn1_Plus FIM	p28	1.000e-04	9.552e-05	3.307e-06	8.904e-05	1.020e-04	4.5
CUQDyn1_Plus FIM	p29	4.000e-04	3.777e-04	1.700e-05	3.443e-04	4.110e-04	5.6
CUQDyn1_Plus HybridCov	p1	0.5	0.144	14.6	-28.4	28.7	71.1
CUQDyn1_Plus HybridCov	p2	0.2	0.502	9.73	-18.6	19.6	151.1
CUQDyn1_Plus HybridCov	p3	0.1	0.155	4.44	-8.55	8.86	55.4
CUQDyn1_Plus HybridCov	p4	1	0.745	1.14	-1.48	2.97	25.5
CUQDyn1_Plus HybridCov	p5	0.1	0.0764	0.108	-0.136	0.288	23.6
CUQDyn1_Plus HybridCov	p6	5.000e-07	1.030e-06	5.225e-04	-0.00102	0.00103	105.9
CUQDyn1_Plus HybridCov	p7	1.000e-04	9.891e-05	3.437e-06	9.218e-05	1.056e-04	1.1
CUQDyn1_Plus HybridCov	p8	4.000e-04	3.923e-04	1.749e-05	3.580e-04	4.266e-04	1.9
CUQDyn1_Plus HybridCov	p9	0.5	0.75	28.8	-55.6	57.1	50.1
CUQDyn1_Plus HybridCov	p10	1.000e-04	5.198e-05	3.384e-04	-6.114e-04	7.153e-04	48.0
CUQDyn1_Plus HybridCov	p11	2.000e-05	3.309e-05	0.0154	-0.0301	0.0302	65.4
CUQDyn1_Plus HybridCov	p12	5.000e-07	1.017e-06	5.161e-04	-0.00101	0.00101	103.4
CUQDyn1_Plus HybridCov	p13	1.000e-04	9.393e-05	3.237e-06	8.759e-05	1.003e-04	6.1
CUQDyn1_Plus HybridCov	p14	4.000e-04	3.695e-04	1.669e-05	3.368e-04	4.022e-04	7.6
CUQDyn1_Plus HybridCov	p15	0.5	0.851	263	-515	517	70.3

Method	Parameter	True	Estimate	StdDev	Lower	Upper	RelErr %
CUQDyn1_Plus HybridCov	p16	3.000e-04	4.106e-04	2.591e-04	-9.729e-05	9.185e-04	36.9
CUQDyn1_Plus HybridCov	p17	0.0025	0.0025	2.324e-05	0.00246	0.00255	0.1
CUQDyn1_Plus HybridCov	p18	0.1	0.0666	20.6	-40.3	40.5	33.4
CUQDyn1_Plus HybridCov	p19	0.0015	2.797e-04	0.102	-0.199	0.2	81.4
CUQDyn1_Plus HybridCov	p20	2.500e-05	2.594e-05	5.626e-07	2.484e-05	2.704e-05	3.8
CUQDyn1_Plus HybridCov	p21	1.250e-04	1.292e-04	3.830e-06	1.217e-04	1.367e-04	3.4
CUQDyn1_Plus HybridCov	p22	5	5.64	6.92	-7.92	19.2	12.8
CUQDyn1_Plus HybridCov	p23	0.0025	3.644e-04	0.0732	-0.143	0.144	85.4
CUQDyn1_Plus HybridCov	p24	0.01	0.00202	0.0624	-0.12	0.124	79.8
CUQDyn1_Plus HybridCov	p25	0.001	0.0018	0.0739	-0.143	0.147	80.5
CUQDyn1_Plus HybridCov	p26	5.000e-04	0.00184	0.00922	-0.0162	0.0199	267.0
CUQDyn1_Plus HybridCov	p27	5.000e-07	1.025e-06	5.202e-04	-0.00102	0.00102	105.0
CUQDyn1_Plus HybridCov	p28	1.000e-04	9.570e-05	3.316e-06	8.920e-05	1.022e-04	4.3
CUQDyn1_Plus HybridCov	p29	4.000e-04	3.785e-04	1.704e-05	3.451e-04	4.119e-04	5.4

S18 Full Trajectory-Band Table

S18.1 LV

Method	State	Mean band width	Median band width	Obs. coverage %	Obs. RMSE
PyMC ABC-SMC	prey	10.9	10.5	96.8	2.58
PyMC ABC-SMC	predator	15.7	3.53	–	–
CUQDyn1_Plus FIM	prey	15	15.4	100.0	2.33
CUQDyn1_Plus FIM	predator	13.3	3.03	–	–
CUQDyn1_Plus HybridCov	prey	15	15.4	100.0	2.33
CUQDyn1_Plus HybridCov	predator	13.4	2.79	–	–

S18.2 SIR

Method	State	Mean band width	Median band width	Obs. coverage %	Obs. RMSE
PyMC ABC-SMC	susceptible	18.8	9.82	–	–
PyMC ABC-SMC	infected	43.6	41.1	100.0	10
PyMC ABC-SMC	recovered	16.3	11.2	–	–
CUQDyn1_Plus FIM	susceptible	9.85	4.95	–	–
CUQDyn1_Plus FIM	infected	43.5	44.9	100.0	10
CUQDyn1_Plus FIM	recovered	9.05	6.06	–	–
CUQDyn1_Plus HybridCov	susceptible	8.64	4.14	–	–

Method	State	Mean band width	Median band width	Obs. coverage %	Obs. RMSE
CUQDyn1_Plus HybridCov	infected	43.5	44.9	100.0	10
CUQDyn1_Plus HybridCov	recovered	9.17	6.09	–	–

S18.3 AP

Method	State	Mean band width	Median band width	Obs. coverage %	Obs. RMSE
PyMC ABC-SMC	y1	41.5	38.7	100.0	8.81
PyMC ABC-SMC	y2	26.6	29.7	100.0	4.39
PyMC ABC-SMC	y3	5.17	5.21	100.0	0.93
PyMC ABC-SMC	y4	1.52	1.46	88.9	0.351
PyMC ABC-SMC	y5	20.4	24.9	–	–
CUQDyn1_Plus FIM	y1	37.8	42.6	100.0	8.17
CUQDyn1_Plus FIM	y2	20.5	23.1	100.0	3.54
CUQDyn1_Plus FIM	y3	3.2	3.54	100.0	0.569
CUQDyn1_Plus FIM	y4	2.95	3.09	100.0	0.403
CUQDyn1_Plus FIM	y5	9.93	11.7	–	–
CUQDyn1_Plus HybridCov	y1	37.8	42.6	100.0	8.17
CUQDyn1_Plus HybridCov	y2	20.5	23.1	100.0	3.54
CUQDyn1_Plus HybridCov	y3	3.2	3.54	100.0	0.569
CUQDyn1_Plus HybridCov	y4	2.95	3.09	100.0	0.403
CUQDyn1_Plus HybridCov	y5	10	13.1	–	–

S18.4 NFKB

Method	State	Mean band width	Median band width	Obs. coverage %	Obs. RMSE
PyMC ABC-SMC	y1	0.0538	0.0505	100.0	0.0114
PyMC ABC-SMC	y2	7.794e-04	7.381e-04	100.0	1.924e-04
PyMC ABC-SMC	y3	0.0858	0.0874	100.0	0.0149
PyMC ABC-SMC	y4	0.0227	0.0208	–	–
PyMC ABC-SMC	y5	5.402e-04	5.078e-04	100.0	1.290e-04
PyMC ABC-SMC	y6	5.109e-06	6.596e-08	–	–
PyMC ABC-SMC	y7	8.389e-06	7.901e-06	91.9	2.001e-06
PyMC ABC-SMC	y8	1.15e+03	1.2e+03	–	–
PyMC ABC-SMC	y9	0.152	0.153	100.0	0.0279
PyMC ABC-SMC	y10	1.06e+03	1.08e+03	–	–
PyMC ABC-SMC	y11	883	855	100.0	190
PyMC ABC-SMC	y12	0.15	0.151	100.0	0.0258
PyMC ABC-SMC	y13	0.0125	0.0124	89.2	0.00314

Method	State	Mean band width	Median band width	Obs. coverage %	Obs. RMSE
PyMC ABC-SMC	y14	4.732e-05	3.029e-08	–	–
PyMC ABC-SMC	y15	0.163	0.163	100.0	0.0318
CUQDyn1_Plus FIM	y1	0.00585	0.00599	94.6	0.00129
CUQDyn1_Plus FIM	y2	9.399e-06	9.634e-06	97.3	1.694e-06
CUQDyn1_Plus FIM	y3	0.0338	0.0347	94.6	0.00851
CUQDyn1_Plus FIM	y4	0.00952	0.00878	–	–
CUQDyn1_Plus FIM	y5	1.279e-05	1.313e-05	100.0	1.235e-06
CUQDyn1_Plus FIM	y6	2.227e-04	9.921e-07	–	–
CUQDyn1_Plus FIM	y7	9.526e-06	9.721e-06	94.6	1.989e-06
CUQDyn1_Plus FIM	y8	1.919e+05	2.206e+05	–	–
CUQDyn1_Plus FIM	y9	0.0293	0.0298	97.3	0.00703
CUQDyn1_Plus FIM	y10	5.780e+04	5.515e+04	–	–
CUQDyn1_Plus FIM	y11	86.7	88.9	97.3	21.5
CUQDyn1_Plus FIM	y12	0.0431	0.0445	97.3	0.00924
CUQDyn1_Plus FIM	y13	0.0136	0.014	94.6	0.00316
CUQDyn1_Plus FIM	y14	1.414e-04	1.451e-07	–	–
CUQDyn1_Plus FIM	y15	0.0426	0.0439	94.6	0.00965
CUQDyn1_Plus HybridCov	y1	0.0058	0.00591	94.6	0.00129
CUQDyn1_Plus HybridCov	y2	1.068e-05	1.084e-05	100.0	1.710e-06
CUQDyn1_Plus HybridCov	y3	0.0339	0.0347	94.6	0.00855
CUQDyn1_Plus HybridCov	y4	1.66	1.41	–	–
CUQDyn1_Plus HybridCov	y5	1.068e-05	1.090e-05	100.0	1.229e-06
CUQDyn1_Plus HybridCov	y6	0.00537	4.183e-05	–	–
CUQDyn1_Plus HybridCov	y7	9.550e-06	9.722e-06	94.6	2.000e-06
CUQDyn1_Plus HybridCov	y8	2.917e+05	3.329e+05	–	–
CUQDyn1_Plus HybridCov	y9	0.0375	0.0368	100.0	0.00819
CUQDyn1_Plus HybridCov	y10	6.584e+04	6.297e+04	–	–
CUQDyn1_Plus HybridCov	y11	100	103	97.3	21.5
CUQDyn1_Plus HybridCov	y12	0.044	0.0457	97.3	0.00924
CUQDyn1_Plus HybridCov	y13	0.0145	0.0148	94.6	0.0032
CUQDyn1_Plus HybridCov	y14	0.0152	9.700e-06	–	–
CUQDyn1_Plus HybridCov	y15	0.0432	0.0444	94.6	0.00965

S19 Detailed Discussion of the Updated Comparison

The PyMC workflows used for the shared comparison tables were rerun after harmonizing several settings that materially affect the comparison: LV and SIR now use tighter ODE tolerances inside the ABC simulator; SIR, AP, and the tabulated NF-kB comparison use 1000 SMC draws per chain, while LV uses 4000 draws per chain; AP priors now match the widened CUQDyn AP parameter bounds; and all PyMC examples export 500 trajectory samples. For observed states, the exported PyMC predictive trajectories now include an additive observation-noise term estimated from the RMSE of the posterior mean against the observed data. Hidden-state PyMC bands remain latent parameter-uncertainty bands. An independent follow-up NF-kB PyMC rerun with 4000 draws per chain is discussed below as a convergence sensitivity check, not as a replacement for the tabulated comparison CSVs.

LV. LV shows strong agreement between PyMC and CUQDyn. The mean parameter relative error is 6.2% for PyMC and about 4.8% for CUQDyn FIM, with all four parameters within 20% relative error for all methods. Observed prey coverage is 96.8% for PyMC and 100% for CUQDyn. PyMC gives a narrower observed-state band than CUQDyn, while the hidden predator band is somewhat wider on average. This is a well-behaved case where both methods tell the same practical story.

SIR. SIR is the clearest agreement case. PyMC and CUQDyn recover beta and gamma with mean relative errors below 1%: 0.6% for PyMC and 0.7% for CUQDyn FIM. The infected-state RMSE is essentially identical across methods, and all methods give 100% observed-state coverage. The PyMC hidden-state bands are wider than CUQDyn bands, reflecting posterior sampling plus the ABC tolerance, but the central trajectories and observed-state uncertainty are highly consistent.

AP. AP remains more sensitive to prior/bound choices. After widening the PyMC AP priors to match CUQDyn, PyMC's mean parameter relative error increases to 97.0%, driven mostly by p4 and p5. CUQDyn FIM and HybridCov remain around 23.0% mean relative error. However, the trajectory comparison is much closer than the raw parameter errors suggest: PyMC observed-state coverage averages 97.2%, CUQDyn gives 100%, and the observed-state RMSE values are of the same order. This indicates practical sloppiness/non-identifiability in AP: distinct parameter combinations can generate similar observed y1-y4 dynamics, especially when y5 is hidden.

NF-kB. NF-kB remains the stress test and should not be presented as a clean parameter-recovery success. In the 1000-draw comparison run, PyMC has mean parameter relative error 107.4%, compared with CUQDyn FIM at 45.2% and HybridCov at 54.7%. The current PyMC NF-kB workflow uses the same broad support as CUQDyn rather than a prior centred on the answer, but the 1000-draw run fails the standard R-hat/ESS gate. A follow-up 4000-draw rerun improves sampler diagnostics to a marginal pass (max R-hat about 1.01, min bulk ESS about 503), showing that the sampling issue can be reduced by more draws. However, the posterior means remain close to the broad-prior mean, approximately two times the nominal values for many rates. Thus convergence does not imply identifiability: these parameter summaries are diagnostic of weak or structural non-identifiability, not trustworthy parameter estimates. CUQDyn gives much lower observed-state RMSE on average, whereas PyMC's observed RMSE is dominated by y11. After noise augmentation, PyMC observed-state coverage is high (98.1%), but some hidden-state PyMC bands are very wide, especially y8, y10, and y11-related dynamics. CUQDyn hidden-state bands should be interpreted together with the FIM rank and reliability diagnostics in this high-dimensional, partially

observed system.

Overall conclusion. For LV and SIR, the methods agree closely in both parameter recovery and predictive behavior. For AP, the predictive agreement is better than the parameter agreement, which is consistent with partial observation and parameter compensation. For NF-kB, the 1000-draw PyMC comparison run is not fully converged; a 4000-draw sensitivity rerun can pass the R-hat/ESS gate marginally, but the posterior remains prior-dominated. NF-kB rate constants should therefore not be reported as recovered. The comparison is most meaningful at the level of observed trajectory fit, posterior predictive coverage, FIM reliability diagnostics, convergence diagnostics, and qualitative hidden-state uncertainty rather than one-to-one parameter intervals.

Part IV. Additional diagnostics and reproducibility

S20 UQ identifiability diagnostics

Post-hoc diagnostics from `diagnose_uq_quality` (`uq_diagnostics.xlsx`), generated for every main run. cond(Cov_p) is the parameter-covariance condition number; unreliable bands are hidden states whose delta-method variance projects strongly onto weak FIM directions.

Table S21: Parameter-covariance conditioning and weak-direction reliability across all main runs.

Model	Method	n_p	cond(Cov)	$\lambda_{\min}$	$\lambda_{\max}$	#neg	#tiny	#unrel	max weak
LV	FIM	4	9.006e+04	1.941e-08	0.001748	0	0	0	0
LV	HybridCov	4	1.594e+05	1.095e-08	0.001745	0	0	0	0
SIR	FIM	2	1.726e+05	2.704e-10	4.667e-05	0	0	0	0
SIR	HybridCov	2	2.073e+05	2.251e-10	4.667e-05	0	0	0	0
AP	FIM	5	8762	1.055e-11	9.245e-08	0	0	0	0
AP	HybridCov	5	2.299e+05	4.106e-13	9.440e-08	0	1	0	0
NFKB	FIM	29	3.073e+18	9.266e-15	2.848e+04	0	4	0	0
NFKB	HybridCov	29	3.100e+19	-6.304e-12	6.965e+04	0	4	0	0

S21 HybridCov covariance diagnostics

HybridCov replaces the FIM parameter correlation with the LOO empirical correlation while keeping the FIM marginal scale. The figures below (produced for the low-dimensional LV and SIR runs) show the LOO correlation matrix, the FIM-vs-LOO marginal standard deviations, and the resulting hidden-state band comparison.

S21.1 LV

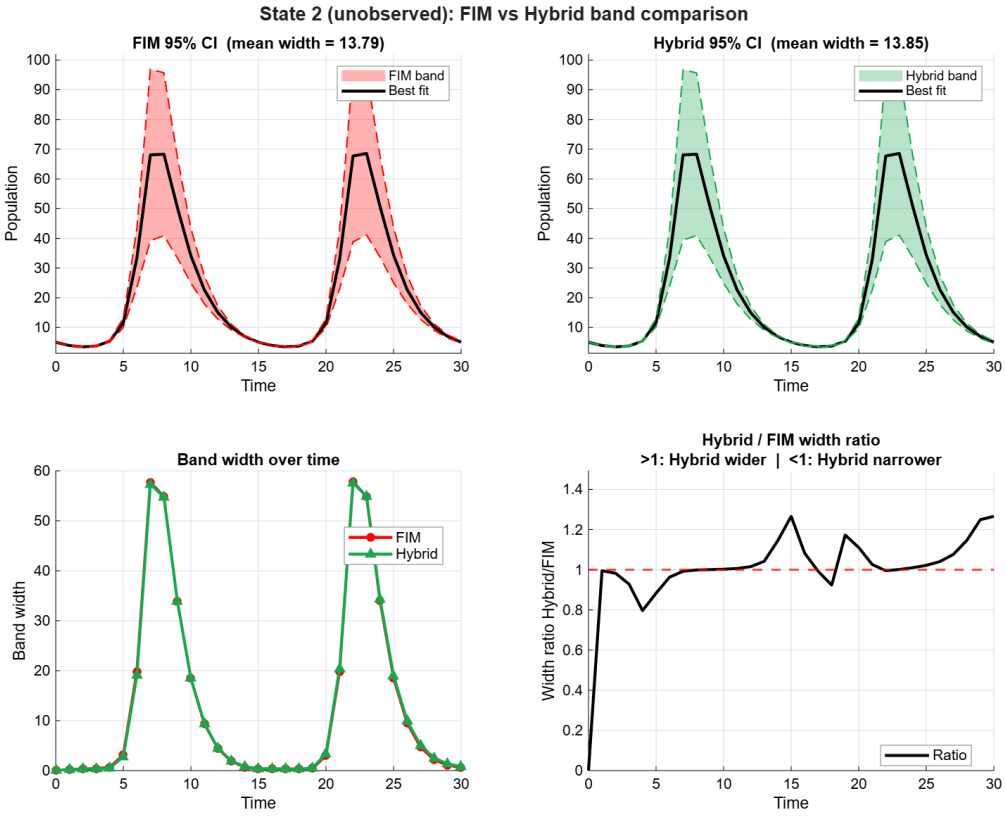

Figure S32: LV HybridCov diagnostic: band comparison state2.

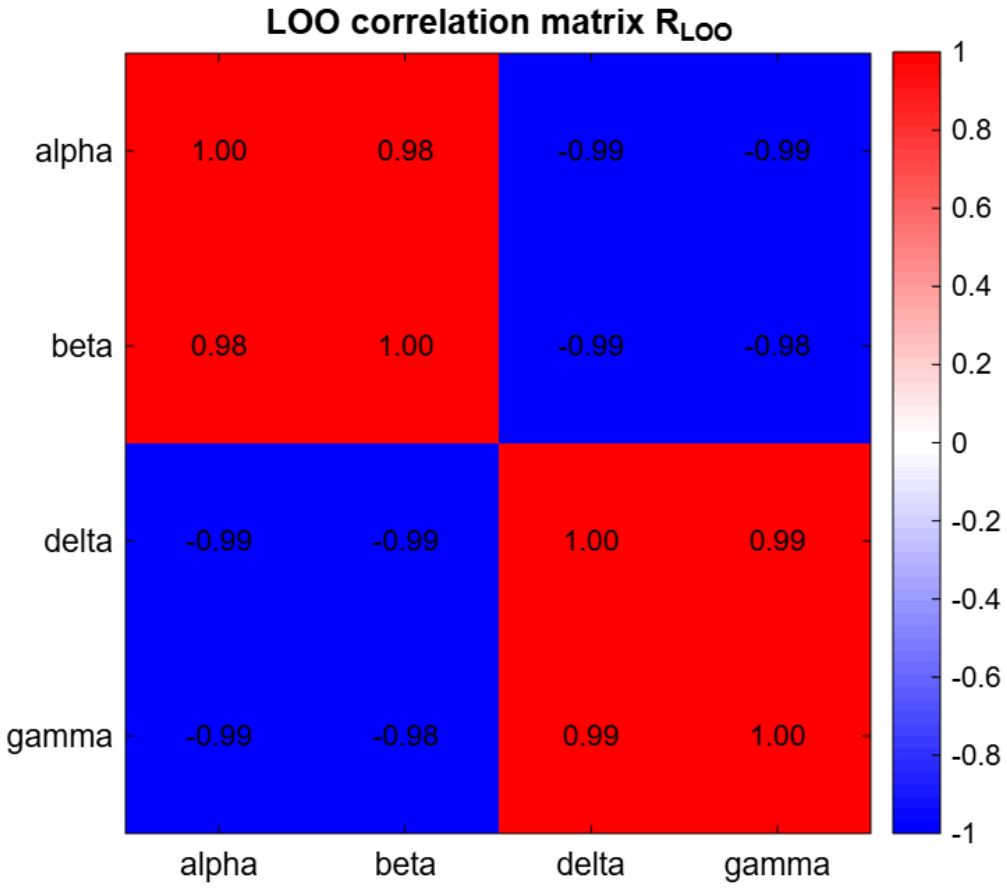

Figure S33: LV HybridCov diagnostic: loo correlation.

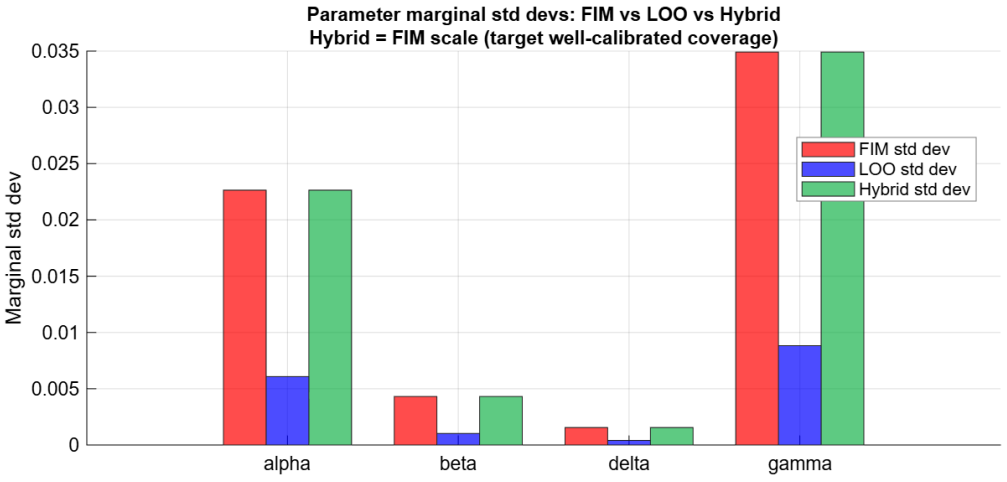

Figure S34: LV HybridCov diagnostic: marginal stddev.

S21.2 SIR

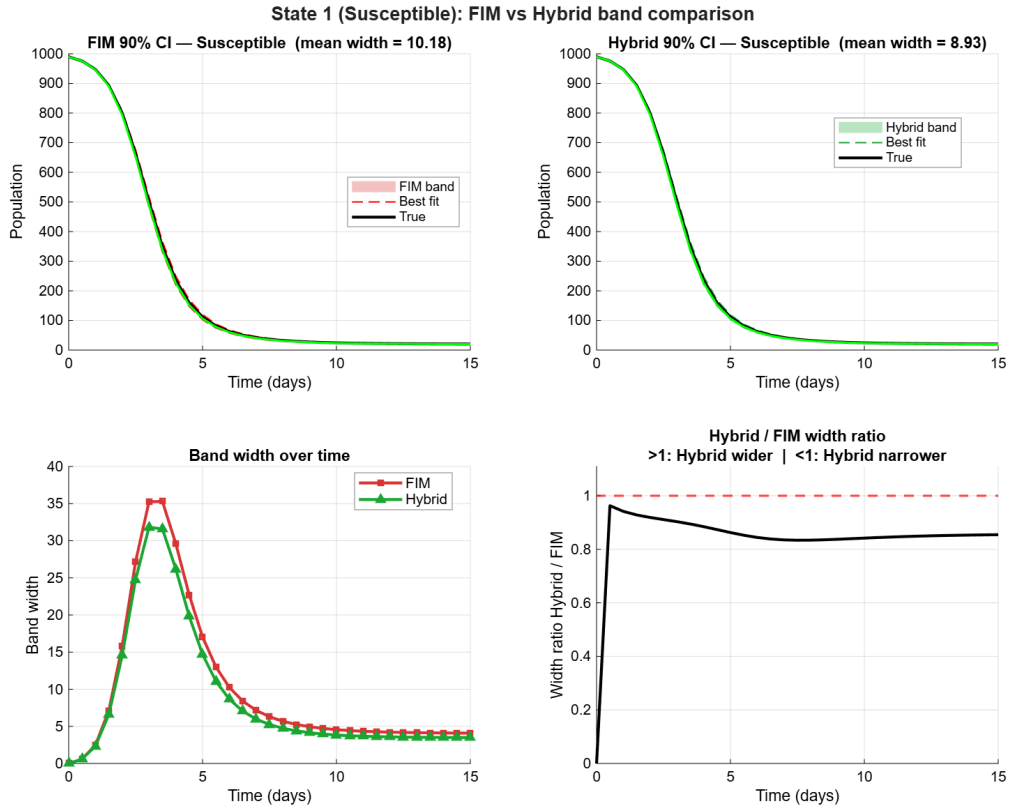

Figure S35: SIR HybridCov diagnostic: band comparison state1.

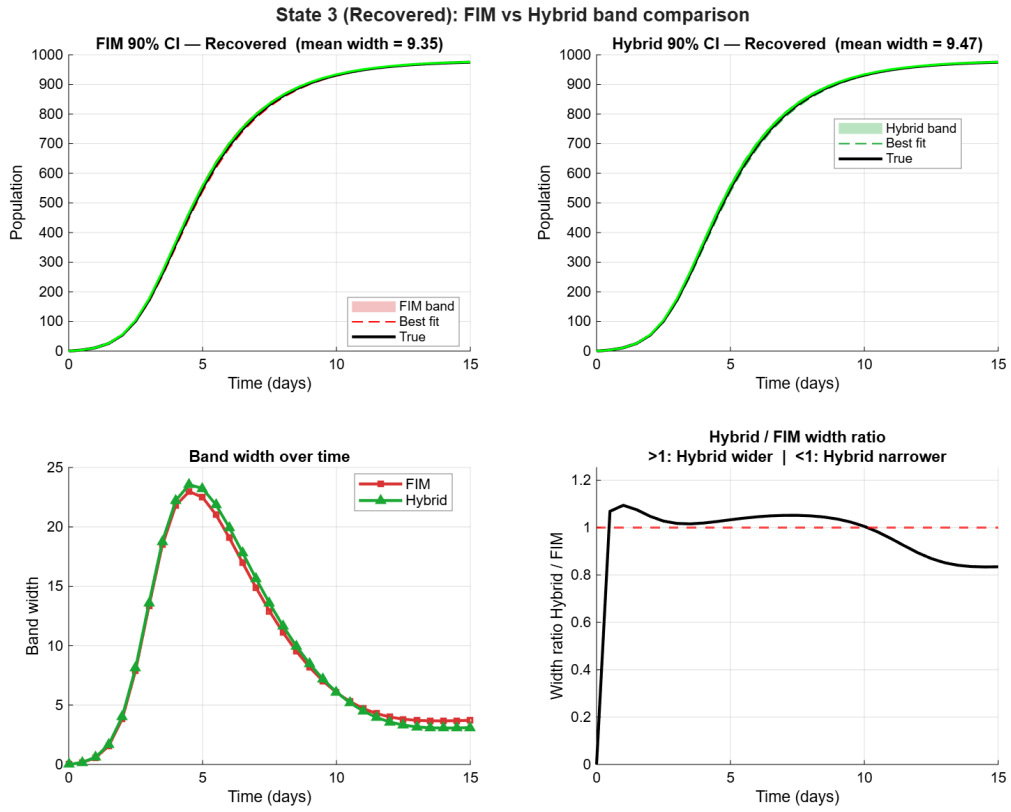

Figure S36: SIR HybridCov diagnostic: band comparison state3.

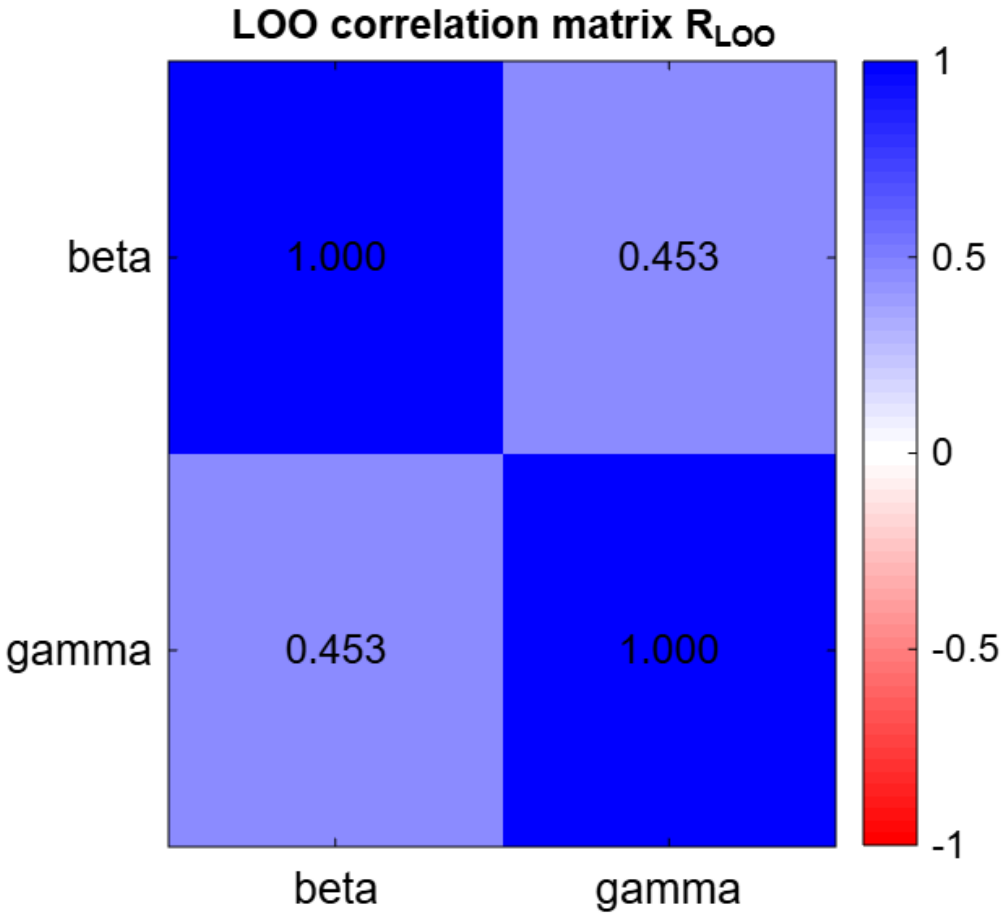

Figure S37: SIR HybridCov diagnostic: loo correlation.

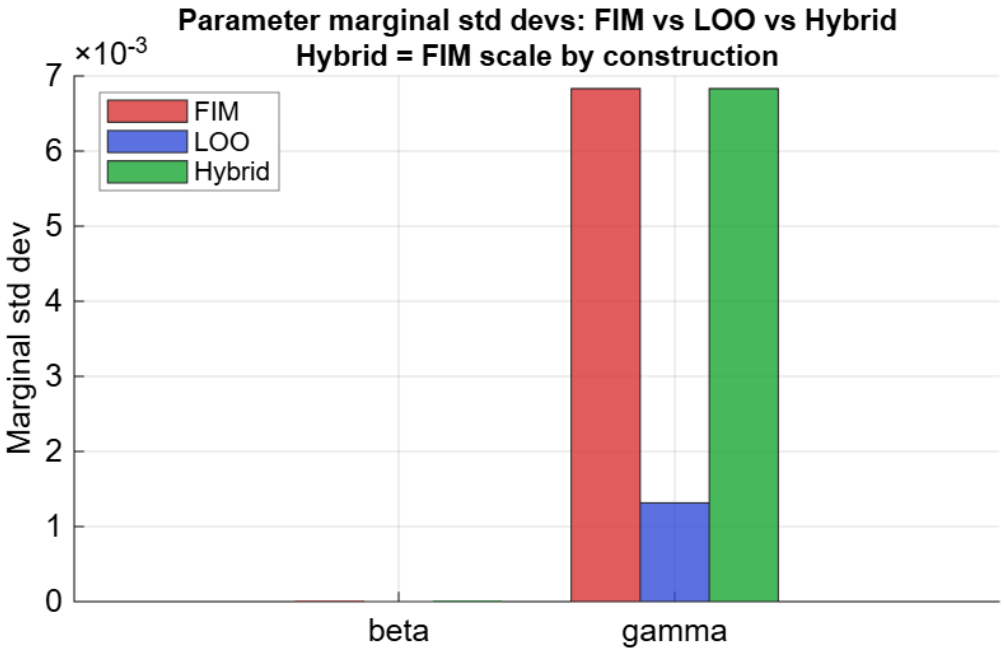

Figure S38: SIR HybridCov diagnostic: marginal stddev.

S22 A non-identifiability failure mode revealed by SBC (LinearCascade3)

LinearCascade3 observes only the terminal state x_3. Because $x_3(t)$ depends solely on the symmetric functions of the cascade poles $\{k_1, k_2, k_3\}$, the inverse problem has an exact $k_1 \leftrightarrow k_2$ permutation symmetry: the parameter sets (k_1, k_2, k_3) and (k_2, k_1, k_3) give numerically identical observed trajectories ($\|x_3(k_1, k_2, k_3) - x_3(k_2, k_1, k_3)\| \approx 10^{-9}$) but different hidden x_1, x_2. The parameter bounds pin k_3 as the slowest pole, leaving a clean two-fold degeneracy.

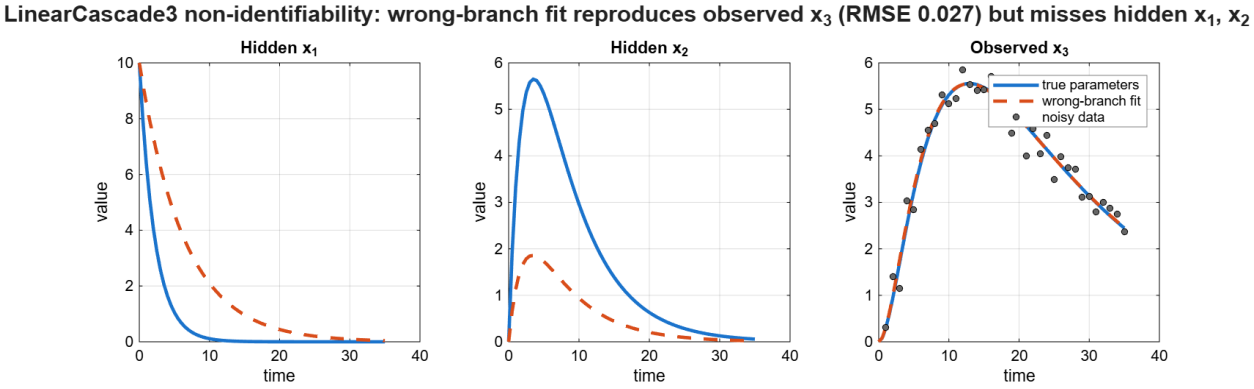

Figure S39: LinearCascade3 upstream non-identifiability. A wrong-branch fit (fitted parameters from a 0%-coverage SBC replicate) reproduces the observed x_3 essentially exactly, yet gives very different hidden x_1 and x_2 trajectories, so the delta-method hidden-state bands are centered on the wrong trajectory.

This produces *bimodal* SBC coverage. Over 50 shared-fit replicates the optimizer recovers the correct branch on ∼34 replicates (≈100% coverage) and the swapped branch on ∼14 (≈0% coverage), giving a mean pointwise coverage of ≈74% at the nominal 95% level (a smaller 20-replicate sample happens to draw mostly the correct branch and reports ≈97%). On the failing replicates the wrong-branch parameters yield a *lower* objective than the true parameters, and a multistart initialized at the truth still converges to the wrong branch, so the failure cannot be removed by more optimizer effort.

Notably, this failure is *not* flagged by the FIM weak-direction reliability diagnostic (Table S21): at either branch the local Gauss–Newton covariance is well conditioned ($\mathrm{cond}(\mathrm{Cov}_\theta) \approx 10^4$, no weak directions), because the degeneracy is global and discrete rather than a local flat direction. Local identifiability diagnostics and SBC are therefore complementary: SBC is required to expose global or multimodal non-identifiability that leaves the local FIM well conditioned. FIM and HybridCov behave identically here, since the failure originates in the location of the fitted parameters, not in the covariance shape.

S23 Bayesian ABC-SMC diagnostics

Convergence and posterior summaries for the PyMC ABC-SMC reference. NF-κB exports full SMC diagnostics (`smc_diagnostics.csv`: split-$\hat{R}$ and effective sample sizes); LV/SIR/AP expose pooled posterior draws (`posterior_samples.csv`) from which posterior mean, standard deviation, and central 95% intervals are computed.

Table S22: NF-κB ABC-SMC posterior and convergence diagnostics.

Param	mean	sd	2.5%	97.5%	$\mathrm{ESS}_{\mathrm{bulk}}$	$\mathrm{ESS}_{\mathrm{tail}}$	$\hat{R}$
p[0]	1.055	0.549	0.171	1.968	1133	1673	1.01
p[1]	0.415	0.213	0.081	0.78	1173	1900	1.01
p[2]	0.211	0.106	0.034	0.381	240	1699	1.02
p[3]	2.018	1.037	0.367	3.85	1488	1229	1.01
p[4]	0.203	0.104	0.031	0.38	1424	1143	1.01
p[5]	0	0	0	0	1638	1319	1.01
p[6]	0	0	0	0	1282	1711	1
p[7]	0.001	0	0	0.002	1353	1188	1
p[8]	0.915	0.445	0.196	1.737	1800	1903	1.01
p[9]	0	0	0	0	1334	1546	1
p[10]	0	0	0	0	1882	1712	1.01
p[11]	0	0	0	0	1777	1434	1.01
p[12]	0	0	0	0	126	919	1.04
p[13]	0.001	0	0	0.002	157	1187	1.03
p[14]	1.086	0.524	0.194	1.916	1610	1283	1.01
p[15]	0.001	0	0	0.001	1362	1038	1.01
p[16]	0.006	0.002	0.002	0.01	1601	2379	1.01
p[17]	0.192	0.102	0.025	0.366	237	1051	1.03
p[18]	0.003	0.002	0	0.006	1189	1936	1.01
p[19]	0	0	0	0	1260	1286	1.01
p[20]	0	0	0	0	1560	1306	1.01
p[21]	8.916	5.647	0.828	18.75	1023	1057	1.01
p[22]	0.005	0.003	0.001	0.01	1660	1073	1.01
p[23]	0.021	0.011	0.003	0.038	1707	1720	1.01
p[24]	0.002	0.001	0	0.003	490	799	1.02
p[25]	0.001	0	0	0.002	1619	1376	1.01
p[26]	0	0	0	0	1057	991	1.01
p[27]	0	0	0	0	1812	857	1
p[28]	0.001	0	0	0.002	1570	867	1

Table S23: LV ABC-SMC posterior summary (16000 pooled draws).

Parameter	mean	sd	2.5%	97.5%
alpha	0.5113	0.02804	0.462	0.5718
beta	0.02313	0.005327	0.01408	0.03471
delta	0.01937	0.001869	0.01596	0.02325
gamma	0.4819	0.04229	0.4037	0.5682

Table S24: SIR ABC-SMC posterior summary (4000 pooled draws).

Parameter	mean	sd	2.5%	97.5%
beta	0.001981	2.537e-05	0.001932	0.002031
gamma	0.4987	0.01042	0.4791	0.519

Table S25: AP ABC-SMC posterior summary (4000 pooled draws).

Parameter	mean	sd	2.5%	97.5%
p1	6.811e-05	1.527e-05	4.271e-05	1.029e-04
p2	4.437e-05	1.598e-05	1.845e-05	8.016e-05
p3	1.975e-05	4.129e-06	1.310e-05	2.917e-05
p4	7.802e-04	2.979e-04	2.268e-04	0.001316
p5	1.332e-04	3.995e-05	4.930e-05	1.936e-04

S24 Optimizer, ODE, cost and FIM settings per run

Effective run options recorded in each result folder's `run_options.xlsx`. FIM and HybridCov share the same shared fitting core, so their optimizer/ODE/cost settings match; they differ only in the unobserved-state covariance construction.

Table S26: Effective run settings for the LV runs.

Option	FIM	HybridCov
meigo.maxeval	2000	2000
meigo.log_var	[1 2 3 4]	[1 2 3 4]
meigo.local.solver	lsqnonlin	lsqnonlin
meigo.inter_save	0	0
meigo.iterprint	1	1
refit.strategy	global	global
refit.local_solver	lsqnonlin	lsqnonlin
refit.local_maxeval_factor	100	100
refit.retry_global_on_failure	true	true
refit.max_cost_ratio_from_start	1.01	1.01
refit.max_parameter_fold_change	10	10
refit.bound_tol	1e-06	1e-06
refit.retry_global_on_bound_hit	false	false
ode.solver	ode15s	ode15s
ode.RelTol	1e-06	1e-06
ode.AbsTol	1e-08	1e-08
ode.NonNegative	nan	nan
cost.residual_model	known_sigma	known_sigma
cost.sigma	2.4531	2.4531
cost.sigma_is_known	true	true
cost.sigma_floor	1e-12	1e-12
cost.sigma_mode	explicit	explicit
cost.noise_percent	nan	nan
cost.observed_state_weights	nan	nan
uq.alp	0.025	0.025
fim.parameterization	log	log
fim.covariance_method	relative_ridge	relative_ridge
fim.relative_ridge	1e-12	1e-12
fim.rank_tol_factor	100	100
fim.weak_fraction_threshold	0.1	0.1

Table S27: Effective run settings for the SIR runs.

Option	FIM	HybridCov
meigo.maxeval	1000	1000
meigo.log_var	[1 2]	[1 2]
meigo.local.solver	lsqnonlin	lsqnonlin
meigo.inter_save	0	0
meigo.iterprint	1	1
refit.strategy	global	global
refit.local_solver	lsqnonlin	lsqnonlin
refit.local_maxeval_factor	100	100
refit.retry_global_on_failure	true	true
refit.max_cost_ratio_from_start	1.01	1.01
refit.max_parameter_fold_change	10	10
refit.bound_tol	1e-06	1e-06
refit.retry_global_on_bound_hit	false	false
ode.solver	ode15s	ode15s
ode.RelTol	1e-08	1e-08
ode.AbsTol	1e-08	1e-08
ode.NonNegative	nan	nan
cost.residual_model	known_sigma	known_sigma
cost.sigma	12.9975	12.9975
cost.sigma_is_known	true	true
cost.sigma_floor	1e-12	1e-12
cost.sigma_mode	explicit	explicit
cost.noise_percent	nan	nan
cost.observed_state_weights	nan	nan
uq.alp	0.05	0.05
fim.parameterization	log	log
fim.covariance_method	relative_ridge	relative_ridge
fim.relative_ridge	1e-12	1e-12
fim.rank_tol_factor	100	100
fim.weak_fraction_threshold	0.1	0.1

Table S28: Effective run settings for the AP runs.

Option	FIM	HybridCov
meigo.maxeval	10000	20000
meigo.log_var	[1 2 3 4 5]	[1 2 3 4 5]
meigo.local.solver	lsqnonlin	lsqnonlin
meigo.inter_save	0	0
meigo.iterprint	1	0
refit.strategy	global	global
refit.local_solver	lsqnonlin	lsqnonlin
refit.local_maxeval_factor	100	100
refit.retry_global_on_failure	true	true
refit.max_cost_ratio_from_start	1.01	1.01
refit.max_parameter_fold_change	10	10
refit.bound_tol	1e-06	1e-06
refit.retry_global_on_bound_hit	false	false
ode.solver	ode15s	ode15s
ode.RelTol	1e-06	1e-06
ode.AbsTol	1e-08	1e-08
ode.NonNegative	nan	nan
cost.residual_model	none	none
cost.sigma	nan	nan
cost.sigma_is_known	true	true
cost.sigma_floor	1e-12	1e-12
cost.sigma_mode	explicit	explicit
cost.noise_percent	nan	nan
cost.observed_state_weights	nan	nan
uq.alp	0.05	0.05
fim.parameterization	log	log
fim.covariance_method	relative_ridge	relative_ridge
fim.relative_ridge	1e-12	1e-12
fim.rank_tol_factor	100	100
fim.weak_fraction_threshold	0.1	0.1

Table S29: Effective run settings for the NFKB runs.

Option	FIM	HybridCov
meigo.maxeval	20000	20000
meigo.log_var	1:29	1:29
meigo.local.solver	lsqnonlin	fmincon
meigo.inter_save	0	0
meigo.iterprint	0	0
refit.strategy	global	global
refit.local_solver	lsqnonlin	lsqnonlin
refit.local_maxeval_factor	100	100
refit.retry_global_on_failure	true	true
refit.max_cost_ratio_from_start	1.01	1.01
refit.max_parameter_fold_change	10	10
refit.bound_tol	1e-06	1e-06
refit.retry_global_on_bound_hit	false	false
ode.solver	ode15s	ode15s
ode.RelTol	1e-06	1e-06
ode.AbsTol	1e-08	1e-08
ode.NonNegative	nan	nan
cost.residual_model	known_sigma	known_sigma
cost.sigma	10-element known-sigma vector for observed NF-κB states; exact values are stored in run_options.xlsx	10-element known-sigma vector for observed NF-κB states; exact values are stored in run_options.xlsx
cost.sigma_is_known	true	true
cost.sigma_floor	1e-12	1e-12
cost.sigma_mode	explicit	explicit
cost.noise_percent	nan	nan
cost.observed_state_weights	nan	nan
uq.alp	0.05	0.05
fim.parameterization	log	log
fim.covariance_method	relative_ridge	relative_ridge
fim.relative_ridge	1e-12	1e-12
fim.rank_tol_factor	100	100
fim.weak_fraction_threshold	0.1	0.1

S25 Per-model method notes

Reproduced from each example's `METHOD_NOTES.md`: the rationale for observed-state choice, residual scaling, parameter bounds, and selected UQ workflows.

S25.1 Lotka-Volterra (LV)

The Lotka-Volterra example is the smallest nonlinear benchmark in the repository. Prey is observed at all post-initial time points, while predator is hidden after the initial condition. This creates a compact partially observed setting where the hidden trajectory is dynamically coupled to the measured state.

The bundled dataset is synthetic with additive Gaussian noise set to 10% of the mean reference prey trajectory. The maintained scripts therefore use `known_sigma` residual scaling. This is the statistically coherent choice for this example because the same observation-error standard deviation is used for data generation, parameter estimation, and FIM covariance estimation.

The parameter bounds are `0.2 * true_parameters` to `2.0 * true_parameters`. These bounds are intentionally moderate: wide enough to test global search and parameter coupling, but narrow enough to avoid unrelated predator-prey regimes that are not represented by the bundled data.

The FIM script is the baseline Gaussian delta-method workflow for the hidden predator state. The HybridCov script reuses the same conformal treatment for observed prey and changes only the covariance used for hidden-state propagation. The LV tutorial also runs the bootstrap workflow, making this the clearest example for comparing FIM, HybridCov, and bootstrap trajectory bands side by side.

S25.2 SIR

The SIR example observes the infected population `I` and treats susceptible `S` and recovered `R` as unobserved after the initial condition. This matches a common epidemic-data setting where incidence or prevalence is measured while the remaining compartments must be inferred through the ODE dynamics.

The bundled data are synthetic with additive Gaussian noise set to 10% of the mean reference infected trajectory. The scripts use `known_sigma` residual scaling so that residuals, optimizer cost, and FIM covariance all use the same measurement-error scale.

The bounds are deliberately broad but physically positive: `beta` ranges from `0.0001` to `0.01`, and `gamma` ranges from `0.01` to `2.0`. These ranges cover plausible transmission and recovery rates around the synthetic truth while preventing nonphysical negative rates.

The FIM and HybridCov scripts use the same observed-state conformal bands for `I`. For hidden `S` and `R`, the FIM script uses local Gaussian propagation from the Gauss-Newton covariance, while the HybridCov script keeps the FIM marginal scale and replaces the correlation structure with the LOO parameter correlation. The SBC scripts are the preferred way to judge whether that covariance choice improves hidden-state calibration for a given SIR setting.

S25.3 Alpha-pinene (AP)

The alpha-pinene example is a five-state compartment model fitted to the experimental `AP_measurementData_1_4.csv` dataset. The maintained scripts use states `x1`, `x2`, `x3`, and `x4` as observed states, while `x5` is treated as unobserved after the initial condition.

The default run scripts use unweighted residuals because the experimental dataset does not provide measurement standard deviations. This makes the objective a direct least-squares fit in the

measurement units. If AP synthetic data are used instead, prefer `known_sigma` with one standard deviation per observed state so that optimization and FIM covariance use the same error model as the data generator.

The parameter bounds are broad, from `0.05 * true_parameters` to `5.0 * true_parameters`, because the experimental fit is more weakly constrained than the synthetic examples and the rates have different physical scales. The initial guess is `0.8 * true_parameters`, which gives the global optimizer a plausible starting region without making the example a local-only fit.

Both `CUQDyn1_Plus` and `CUQDyn1_Plus_HybridCov` are useful here. The observed states receive LOO conformal prediction bands. The hidden states receive delta-method bands from either the FIM covariance or the HybridCov covariance. Because AP has only nine time points, the LOO ensemble is small; HybridCov should be interpreted as an empirical covariance variant rather than a guaranteed improvement for this model.

S25.4 NF-κB

The NF-kB example is the largest maintained benchmark, with 15 states and 29 parameters. The synthetic dataset observes 10 selected states after the initial condition and leaves the remaining states hidden. This gives a higher dimensional partially observed problem than LV, SIR, or AP.

The bundled dataset is synthetic with additive Gaussian noise set to 5% of the mean reference trajectory for each observed state. The maintained FIM and HybridCov scripts use `known_sigma` residual scaling, with one scale per observed state, so states with different magnitudes contribute on a comparable measurement-error scale.

The parameter bounds are `0.1 * true_parameters` to `4.0 * true_parameters`. These bounds preserve positive kinetic rates while giving MEIGO enough room to move in the high-dimensional parameter space. The initial guess is `0.8 * true_parameters`, which keeps the example computationally manageable without turning it into a purely local refinement.

The FIM script is the baseline for hidden-state Gaussian propagation. The HybridCov script is especially relevant in this example because many parameters are coupled through partially observed dynamics. The legacy `run_NFKB_example.m` is retained only as a diagnostic runner; new comparisons should use `run_NFKB_example_CUQDyn1plus.m` and `run_NFKB_example_CUQDyn1plus_HybCov.m`.

NF-kB is also the maintained stress test for identifiability diagnostics. Before interpreting hidden-state bands or parameter intervals, inspect `results.diagnostics.fim` for the FIM parameterization, rank, condition number, and weak-direction count, and inspect `results.diagnostics.fim_reliability` or the `FIMReliability` sheet written by `diagnose_uq_quality`. Bands marked as affected by weak FIM directions should be reported as unreliable rather than as ordinary calibrated prediction intervals.

S25.5 LinearCascade

The LinearCascade examples are controlled known-truth diagnostics rather than general-purpose biological examples. They are included to test whether weighted least-squares fitting, FIM covariance, complex-step sensitivities, and hidden-state delta-method propagation agree with independent analytic or matrix-exponential reference calculations.

The two-state diagnostic observes only downstream state `x2`; upstream state `x1` is hidden. The three-state diagnostic observes only downstream state `x3`; upstream states `x1` and `x2` are hidden. This design intentionally tests the hardest direction for latent-state uncertainty: hidden upstream dynamics must be inferred indirectly from downstream measurements.

Both diagnostics generate synthetic noisy observations internally and use `known_sigma` residual scaling. The sigma is computed from the reference trajectory using the same convention as the synthetic-data helpers, so the optimizer objective and FIM covariance are matched to the data-generation noise model.

The parameter bounds are moderate for the two-state cascade (`0.2 * true_parameters` to `3.0 * true_parameters`) and wider for the three-state cascade (`0.15 * true_parameters` to `4.0 * true_parameters`). The wider three-state bounds make the downstream-only case a stronger identifiability and covariance stress test.

The diagnostics primarily exercise `CUQDyn1_Plus` with FIM covariance and then compare its covariance and hidden-state standard deviations against independent reference calculations. The LinearCascade3 SBC scripts additionally compare FIM and HybridCov from shared fits and are useful for checking hidden-state coverage behavior across repeated synthetic datasets.

S26 Coverage and scope limitations

- Simulation-based calibration (SBC) was run for LV, SIR, AP and LinearCascade3. It was *not* run for NF-κB (high state/parameter dimension) or the 2-state LinearCascade; coverage claims for those rest on the per-run diagnostics only.
- Parametric bootstrap trajectory UQ is demonstrated only for the LV tutorial; the other models use the FIM and HybridCov delta-method bands.
- HybridCov covariance-diagnostic figures are emitted only for the low-dimensional LV and SIR runs.
- Observed-state coverage is conformal (distribution-free under exchangeability); unobserved-state coverage is approximate and local, and should be read together with the identifiability diagnostics in Table S21.